

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

UNIHUB

DIRECTOR: IGNACIO DÍAZ CANO

AUTOR: ALEJANDRO RAMOS RODRÍGUEZ

Puerto Real, Cádiz – 2026

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

UNIHUB

DIRECTOR: IGNACIO DÍAZ CANO

AUTOR: ALEJANDRO RAMOS RODRÍGUEZ

Puerto Real, Cádiz – 2026

Declaración personal de autoría

Alejandro Ramos Rodríguez, estudiante del Grado en Ingeniería Informática en la Escuela Superior de Ingeniería de la Universidad de Cádiz, como autor de este documento académico titulado *UniHub* y presentado como Trabajo Final de Grado.

DECLARO QUE

Es un trabajo original, que no copio ni utilizo parte de obra alguna sin mencionar de forma clara y precisa su origen tanto en el cuerpo del texto como en su bibliografía y que no empleo datos de terceros sin la debida autorización, de acuerdo con la legislación vigente. Asimismo, declaro que soy plenamente consciente de que no respetar esta obligación podrá implicar la aplicación de sanciones académicas, sin perjuicio de otras actuaciones que pudieran iniciarse.

En Puerto Real, a 28 de agosto de 2026

Fdo.: Alejandro Ramos Rodríguez

Agradecimientos

Deseo expresar mi más sincero agradecimiento a la Escuela Superior de Ingeniería (ESI) y al cuerpo docente de la Universidad de Cádiz (UCA) por la formación recibida a lo largo de los estudios del Grado en Ingeniería Informática.

A mi familia y compañeros por su apoyo incondicional durante todo el proceso de desarrollo e investigación del presente Trabajo Fin de Grado.

Resumen

El presente Trabajo Fin de Grado aborda el diseño, desarrollo e implementación de **UniHub**, un sistema informático distribuido e integral para la centralización, almacenamiento, consulta y gestión de la oferta de educación superior en España (universidades públicas y privadas, titulaciones de Grado, Máster y Doctorado, y planes de estudio desglosados del Boletín Oficial del Estado - BOE).

El proyecto se estructura en cuatro fases de ingeniería perfectamente delimitadas e interconectadas:

1. **Fase 1 (Rastreador / Crawler):** Desarrollado en Python, obtiene el catálogo oficial, filtra titulaciones vigentes y analiza las resoluciones del BOE. El análisis de PDF construye un modelo de páginas, posiciones y tablas para delimitar cada titulación y extraer su plan curricular. La ejecución registra su trazabilidad, respeta `robots.txt` en las fuentes web y admite una planificación Cron configurable mediante `CRAWLER_CRON_SCHEDULE`.
2. **Fase 2 (Base de Datos PostgreSQL y API REST):** Modelo relacional DDL con rol de solo lectura de API (`unihub_api_user`), pipeline de carga ETL, ordenación prioritaria (públicas primero, luego privadas), soporte de métricas físicas individuales de contenedores cgroup y servicio REST en FastAPI.
3. **Fase 3 (Portal Web Frontend):** Aplicación Web SPA moderna creada con React y Vite bajo la identidad visual de la Universidad de Cádiz (UCA). Incluye paginación configurable (5, 10, 20, 50 y 100 por página), ocultación de códigos internos, geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual `/admin`.
4. **Fase 4 (Contenerización y Orquestación Docker):** Sistema de 4 contenedores aislados (`unihub_db`, `unihub_crawler`, `unihub_api`, `unihub_www`) orquestados con Docker Compose con garantía de persistencia permanente de datos tras apagar o reiniciar los servicios.

Palabras clave: UniHub, Universidad de Cádiz (UCA), Rastreador Python, BOE, Doctorado, PostgreSQL, FastAPI, React, Geolocalización Haversine, Docker Compose.

Abstract

This Bachelor's Thesis addresses the design, development, and implementation of **UniHub**, a comprehensive, distributed computer system for the centralization, storage, consultation, and management of higher education offerings in Spain (public and private universities, Bachelor's, Master's, and Doctoral degrees, and detailed study plans extracted from the Official State Gazette – BOE).

The project is structured into four clearly delimited and interconnected engineering phases:

1. **Phase 1 (Web Crawler & Extraction):** Developed in Python, it retrieves the official catalog, filters active degrees, and processes BOE resolutions. The PDF deep analysis module constructs a geometric model of pages, text positions, and tabular boundaries to isolate each degree and extract its curricular syllabus. Execution ensures end-to-end traceability, strictly adheres to `robots.txt` policies across institutional sources, and supports automated Cron scheduling via `CRAWLER_CRON_SCHEDULE`.
2. **Phase 2 (PostgreSQL Database & REST API):** Relational DDL schema with a restricted read-only role (`unihub_api_user`), bulk ETL ingestion pipeline, priority sorting (public institutions first, followed by private ones), real-time cgroup container performance telemetry, and a high-performance RESTful service built with FastAPI.
3. **Phase 3 (Web Frontend Portal):** A modern Single Page Application (SPA) built with React and Vite following the corporate visual identity of the University of Cadiz (UCA). It features configurable pagination (5, 10, 20, 50, and 100 items per page), masking of internal database identifiers, Haversine geolocation with distance computed directly onto university cards, a featured universities section (top 6 most visited), an institutional “About Us” section (Bachelor's Thesis by Alejandro Ramos Rodríguez, UCA), and an isolated administrative control panel accessible via the manual URL route `/admin`.
4. **Phase 4 (Docker Containerization & Orchestration):** Multi-container architecture composed of four isolated services (`unihub_db`, `unihub_crawler`, `unihub_api`, `unihub_www`) orchestrated with Docker Compose, providing verified data persistence across container shutdowns and restarts.

Keywords: UniHub, University of Cadiz (UCA), Python Crawler, BOE, Doctorate, PostgreSQL, FastAPI, React, Haversine Geolocation, Docker Compose.

Índice general

Índice de figuras	XII
-------------------	-----

Índice de tablas	XIII
------------------	------

I Prolegómeno	1
----------------------	----------

1. Introducción	2
------------------------	----------

1.1. Motivación	2
1.2. Alcance y Objetivos	2
1.3. Glosario de términos	3
1.4. Organización del documento	3

2. Antecedentes	5
------------------------	----------

2.1. Contexto y justificación del proyecto	5
2.2. Estado de la técnica: Extracción y agregación de datos web	6
2.2.1. Fundamentos teóricos del Web Scraping y la Extracción de Datos	6
2.2.2. Evolución de las técnicas de extracción web	7
2.2.3. Políticas de cortesía, ética y gobernanza en el rastreo web	7
2.3. Análisis de plataformas y alternativas existentes	8
2.4. Aportación y valor diferencial de UniHub	9

3. Plan de gestión de proyecto	11
---------------------------------------	-----------

3.1. Metodología de desarrollo	11
3.2. Tecnologías empleadas	11
3.3. Planificación temporal y diagrama de Gantt	12
3.4. Presupuesto y desglose de costes	13
3.5. Gestión de riesgos	13
3.6. Aseguramiento de la calidad y gestión de la configuración	15

II Desarrollo	16
----------------------	-----------

4. Requisitos del Sistema	17
----------------------------------	-----------

4.1. Objetivos de Negocio	17
4.2. Objetivos del Sistema	17
4.3. Requisitos funcionales	17
4.4. Requisitos no funcionales	19
4.5. Requisitos de información	19
4.6. Reglas de negocio	19
4.7. Análisis GAP	20

5. Análisis del Sistema	23
--------------------------------	-----------

5.1.	Modelo Conceptual y de Dominio	23
5.2.	Modelo de Casos de Uso	23
5.3.	Modelo de Comportamiento Dinámico	25
5.3.1.	Modelo de Comportamiento del Rastreador y Validación Curricular (Fase 1)	25
5.3.2.	Modelo de Secuencia: Simulación Financiera en Calculadora de Matrícula	26
5.4.	Modelo de Interfaz de Usuario	27
6.	Diseño del Sistema	29
6.1.	Arquitectura del Sistema	29
6.1.1.	Diseño de alto nivel	29
6.1.2.	Diseño detallado	29
6.1.3.	Seguridad y Control de Acceso (RBAC)	31
6.2.	Parametrización del software base	31
6.3.	Diseño Físico de Datos	31
6.4.	Diseño de la Interfaz de Usuario	32
6.4.1.	Vistas Principales y Prototipado del Sistema	32
6.4.2.	Panel de Control y Administración Aislado	35
7.	Construcción del Sistema	39
7.1.	Entorno de Construcción	39
7.2.	Código Fuente y Estructura Modular	39
7.2.1.	Fase 1: Módulo de Ingesta y Crawler (Codigo/Crawler/)	39
7.2.2.	Fase 2: Módulo de API REST y Persistencia (Codigo/API/)	47
7.2.3.	Fase 3: Módulo de Interfaz Web y Calculadora (Codigo/WWW/)	48
7.2.4.	Fase 4: Despliegue Contenerizado y Orquestación (Codigo/Docker/)	50
7.2.5.	Pruebas y Auditoría de Rendimiento (Codigo/Pruebas/)	50
7.3.	Esquema Físico Relacional y Modelo de Persistencia	50
8.	Pruebas del Sistema	53
8.1.	Estrategia de Pruebas	53
8.2.	Pruebas Unitarias y de Integración	53
8.3.	Validación del Parser de Resoluciones BOE	54
8.3.1.	Evaluación Empírica Reproducible	54
8.4.	Pruebas de Rendimiento y Benchmarking	55
8.5.	Pruebas de Ejecución Controlada	55
8.6.	Pruebas No Funcionales	56
8.7.	Criterio de Aceptación	56
9.	Despliegue del Sistema	59
9.1.	Arquitectura Física	59
9.2.	Instrucciones de despliegue	59
9.2.1.	Requisitos previos	59
9.2.2.	Inventario de componentes	59
9.2.3.	Procedimientos de instalación	60
9.2.4.	Pruebas de implantación	61

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio	61
III Epílogo	62
10. Conclusiones	63
10.1. Objetivos alcanzados	63
10.1.1. Resultados Cuantitativos y Métricas Clave	63
10.2. Lecciones aprendidas	64
10.3. Trabajo futuro	64
Información sobre Licencia	67
Bibliografía	69
IV Anexos	71
A. Manual del desarrollador	72
A.1. Introducción	72
A.2. Preparación del entorno de trabajo	72
A.3. Consideraciones generales sobre el desarrollo	72
A.4. Instrucciones para construcción	73
A.5. Ejecución Avanzada y Filtrado del Crawler (Fase 1)	73
B. Manual de usuario	77
B.1. Introducción	77
B.2. Instalación	77
B.3. Uso del sistema	77
C. Glosario de Términos y Acrónimos Técnicos	81
C.1. Glosario de Conceptos y Patrones de Diseño	81
D. Esquema Físico DDL de Base de Datos	85

Índice de figuras

3.1. Diagrama de Gantt del proyecto UniHub (Planificación de 450 horas en 16 semanas)	13
5.1. Diagrama de Clases del Modelo de Dominio de UniHub (UML)	24
5.2. Diagrama General de Casos de Uso de UniHub (UML)	24
5.3. Diagrama de Actividad: Ingesta Inteligente y Validación Matemática de Completitud Curricular	25
5.4. Diagrama de Secuencia UML: Interacción en la Simulación Financiera de Matrícula (Fase 3)	26
6.1. Interfaz de usuario: Pantalla de inicio y catálogo interactivo de universidades con buscador y filtros (Fase 3)	33
6.2. Interfaz de usuario: Modal de detalle curricular con asignaturas, créditos ECTS, cuatrimestres y trazabilidad BOE	34
6.3. Interfaz de usuario: Estimador interactivo de liquidación de matrícula con repeticiones y exenciones sociales	35
6.4. Interfaz de usuario: Búsqueda y ordenación de universidades por proximidad geográfica mediante geolocalización	36

Índice de tablas

2.1. Comparativa entre plataformas existentes de información universitaria y UniHub	8
3.1. Distribución de carga de trabajo por fases del proyecto (Total: 450 horas)	12
3.2. Presupuesto detallado del proyecto UniHub (450 horas de trabajo) . . .	14

Parte I

Prolegómeno

1. Introducción

A continuación, se describe la motivación del proyecto **UniHub** y su alcance. También se incluye un glosario de términos y la organización del resto de la presente documentación.

1.1. Motivación

El catálogo de la oferta universitaria de enseñanza superior en España almacena la información de titulaciones oficiales. Sin embargo, la consulta integrada de universidades públicas y privadas, la verificación de planes de estudio vigentes modificados en el Boletín Oficial del Estado (BOE) y la clasificación de Grados, Másteres y Doctorados resultaba compleja y fragmentada.

La motivación principal de este proyecto es construir **UniHub**, un sistema informático moderno, robusto y automatizado que rastree, homogeneice, persista y exponga de forma interactiva la oferta universitaria oficial de España, facilitando la visualización de asignaturas, módulos y créditos ECTS cuando la fuente publicada proporcione ese nivel de detalle.

1.2. Alcance y Objetivos

El alcance del proyecto abarca cuatro fases de ingeniería de software independientes pero integradas:

1. **Desarrollo de un Rastreador (Crawler) en Python (Fase 1):** Capaz de obtener los catálogos oficiales, aplicar filtrado estricto de titulaciones vigentes y autorizadas (descartando extinguidas o no vigentes), clasificar Doctorados, inspeccionar todos los candidatos a PDF del BOE, aplicar resiliencia ante fallos de conexión (pausas de 5 min y cortocircuito a los 15 min), registrar métricas y notificar a la API REST tras completar la recolección.
2. **Diseño de Base de Datos y API REST en Python (Fase 2):** Creación de un esquema relacional en PostgreSQL con control de acceso por roles (`ruct_api_user` de solo lectura), pipeline de ingesta ETL, ordenación prioritaria (públicas primero, luego privadas) y construcción de una API REST en FastAPI con métricas físicas de contenedores cgroup.
3. **Desarrollo del Portal Web Interactivo (Fase 3):** Aplicación SPA en Vite + React con el sistema de diseño visual de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100 elementos por página), geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG

Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual /admin.

4. **Sistemas de Contenerización y Orquestación Docker (Fase 4):** Despliegue independiente de 4 contenedores (`ruct_db`, `ruct_crawler`, `ruct_api`, `ruct_www`) en una red privada virtualizada con persistencia garantizada de datos.

1.3. Glosario de términos

UniHub: Nombre oficial de la plataforma web y sistema distribuido objeto de este Trabajo Fin de Grado.

RUCT: Registro oficial de origen de datos de Universidades, Centros y Títulos.

BOE: Boletín Oficial del Estado.

ECTS: European Credit Transfer and Accumulation System (Sistema Europeo de Transferencia y Acumulación de Créditos).

API REST: Representational State Transfer Application Programming Interface.

SPA: Single Page Application (Aplicación de Página Única).

UCA: Universidad de Cádiz.

ETL: Extract, Transform and Load (Extracción, Transformación y Carga).

Web Vitals: Métricas de rendimiento de la experiencia de usuario en la web (FCP, LCP, TTFB).

Haversine: Fórmula matemática para calcular distancias entre dos puntos en una esfera a partir de sus coordenadas geográficas.

1.4. Organización del documento

La presente memoria se estructura en las siguientes secciones principales:

- **Prolegómeno:** Introducción, antecedentes del dominio universitario y plan de gestión de proyecto.
- **Desarrollo:** Catálogo de requisitos, análisis UML, diseño arquitectónico (patrón C4/capas), implementación en código de cada fase, pruebas ejecutadas y despliegue del sistema Docker.
- **Epílogo:** Conclusiones generales, lecciones aprendidas y líneas de trabajo futuro.
- **Anexos:** Manuales de usuario, guía de desarrollador y ejemplos de tablas e imágenes.

2. Antecedentes

En este capítulo se detalla la situación actual que motiva el diseño e implementación del sistema **UniHub**. Se expone la problemática de dispersión y heterogeneidad de los datos en la educación superior española, se fundamenta teóricamente el empleo de técnicas de rastreo y extracción web (*Web Scraping*) a partir de la literatura científica reciente, se analizan las alternativas tecnológicas existentes y se define el posicionamiento innovador del presente proyecto.

2.1. Contexto y justificación del proyecto

El panorama universitario español se encuentra inmerso en un proceso continuo de transformación y actualización normativa, regulado por la Ley Orgánica 2/2023 del Sistema Universitario (LOSU) [Jefatura del Estado \(2023\)](#) y los Reales Decretos que ordenan las enseñanzas oficiales: el Real Decreto 822/2021 [Ministerio de Universidades \(2021\)](#) para estudios de Grado y Máster, y el Real Decreto 99/2011 [Ministerio de Educación \(2011\)](#) para los programas de Doctorado. Con más de 90 universidades activas entre instituciones públicas y privadas, distribuidas a lo largo de las 17 comunidades autónomas, el catálogo estatal supera las 4.000 titulaciones vigentes.

A pesar de que las administraciones públicas promueven la transparencia y la apertura de datos gubernamentales (*Open Government Data*) [Attard et al. \(2015\)](#); [Janssen et al. \(2012\)](#), en la práctica existe una notable fragmentación informativa:

1. **Dispersión de fuentes jurídicas y académicas:** La aprobación y verificación oficial de cada plan de estudios se publica en el Boletín Oficial del Estado (BOE) o en los diarios oficiales de las comunidades autónomas en formato PDF plano. Por su parte, el catálogo general se registra administrativamente en el Registro de Universidades, Centros y Títulos (RUCT), cuya consulta pública se apoya en interfaces tradicionales y volcados en hojas de cálculo (.xls) sin servicios de interconexión modernos.
2. **Heterogeneidad en la publicación universitaria:** Cada universidad mantiene su propia infraestructura web y sistemas de gestión de contenidos (CMS), publicando sus planes docentes bajo esquemas visuales, nomenclaturas y estructuras dispares. Ello impide que los futuros estudiantes o investigadores puedan comparar planes curriculares de forma homogénea.
3. **Opacidad y complejidad en la tarificación de matrículas:** Los precios públicos universitarios varían significativamente entre comunidades autónomas según los grados de experimentalidad, los recargos por repetición de matrícula (de 1^a a 4^a convocatoria) y los regímenes de exención social (Familias Numerosas, Becas del Ministerio, discapacidad o bonificaciones autonómicas por rendimiento académico). Asimismo, las universidades privadas operan con tarifas anuales

independientes que no suelen estar consolidadas en ninguna plataforma pública de comparación.

4. **Naturaleza diferenciada de las enseñanzas de Doctorado:** Conforme al Real Decreto 99/2011 [Ministerio de Educación \(2011\)](#), los estudios de doctorado no se estructuran en asignaturas lectivas tradicionales con créditos ECTS, sino en líneas de investigación, actividades formativas transversales y tutela académica de la tesis doctoral. La inmensa mayoría de portales existentes cometen el error de tratar los doctorados bajo el prisma de asignaturas lectivas o, por el contrario, los omiten por completo de sus motores de búsqueda.

Ante este escenario, **UniHub** se concibe como una solución integral de ingeniería que centraliza, normaliza, enriquece y expone la totalidad de la oferta universitaria española mediante una arquitectura desacoplada en cuatro fases operativas.

2.2. Estado de la técnica: Extracción y agregación de datos web

La captura, estructuración y consolidación de información proveniente de múltiples fuentes heterogéneas constituye uno de los campos de investigación más prolíficos de la informática aplicada. En esta sección se analiza el estado del arte en extracción de datos web, tecnologías de parsing de documentos semiestructurados y arquitecturas de agregación.

2.2.1. Fundamentos teóricos del Web Scraping y la Extracción de Datos

De acuerdo con las revisiones sistemáticas de Ferrara et al. [Ferrara et al. \(2014\)](#) y Singrodia et al. [Singrodia et al. \(2019\)](#), el proceso de extracción automatizada de datos web (*Web Data Extraction* o *Web Scraping*) consiste en recuperar información no estructurada o semiestructurada de páginas HTML y transformarla en representaciones estructuradas analizables (tales como tablas relacionales o esquemas JSON).

Zhao [Zhao \(2017\)](#) subraya que, a diferencia de los rastreadores web convencionales (*web crawlers*) utilizados por motores de indexación generalista como Google, que buscan recorrer hipervínculos exhaustivamente sin procesar profundamente la semántica del documento, los extractores de datos orientados a dominio combinan el rastreo dirigido (*focused crawling*) con patrones semánticos y reglas heurísticas diseñadas para aislar entidades específicas de interés.

En el ámbito biomédico y científico, Glez-Peña et al. [Glez-Peña et al. \(2014\)](#) destacan que el *web scraping* se ha consolidado como un mecanismo imprescindible para salvar la brecha existente cuando las fuentes primarias no proporcionan Interfaces de Programación de Aplicaciones (APIs) REST documentadas o cuando los servicios web existentes presentan restricciones de volumen o discontinuidad en el servicio.

2.2.2. Evolución de las técnicas de extracción web

La literatura clasifica los mecanismos de extracción web en cuatro grandes familias metodológicas [Ferrara et al. \(2014\)](#); [Crescenzi et al. \(2001\)](#):

- **Generadores de Wrappers y Enfoques Heurísticos / DOM:** Se fundamentan en la navegación sobre el árbol de objetos del documento (*Document Object Model*, DOM) empleando expresiones regulares, selectores XPath o selectores CSS (herramientas como BeautifulSoup, lxml o Scrapy). Este enfoque ofrece una gran velocidad de procesamiento y bajo consumo de recursos computacionales, si bien resulta sensible a cambios estructurales drásticos en las plantillas HTML [Glez-Peña et al. \(2014\)](#).
- **Extracción Inductiva y Reconocimiento de Patrones Repetitivos:** Sistemas pioneros como *RoadRunner* [Crescenzi et al. \(2001\)](#) demostraron la viabilidad de inferir gramáticas y esquemas de extracción comparando automáticamente páginas web generadas a partir de la misma plantilla de base de datos, aislando los datos variables respecto al esqueleto de diseño.
- **Automatización mediante Navegadores Headless (Renderizado Dinámico):** Ante el auge de las aplicaciones web enriquecidas y los frameworks de renderizado en el cliente (*Single Page Applications* con React, Angular o Vue), el contenido textual no reside en el HTML crudo retornado por el servidor, sino que se genera asíncronamente mediante peticiones AJAX o Fetch. En estos entornos, el scraping requiere navegadores sin interfaz gráfica (*Headless Browsers*) tales como Chromium sobre Playwright o Puppeteer, capaces de ejecutar el motor JavaScript V8, evaluar eventos del ciclo de vida de la página y esperar la mutación del DOM antes de volcar el contenido procesado [Singrodia et al. \(2019\)](#).
- **Segmentación y Procesamiento Geométrico de Documentos PDF:** En el contexto institucional de las publicaciones oficiales (BOE), la información legal no reside en etiquetas web sino en flujos vectoriales de documentos PDF. La extracción requiere reconstruir el modelo espacial de caracteres, coordenadas tipográficas, líneas delimitadoras y tablas (*pdfplumber*, *pypdfium2*), combinando en ocasiones algoritmos de Reconocimiento Óptico de Caracteres (OCR) basados en Tesseract cuando los documentos corresponden a escaneos antiguos [Ferrara et al. \(2014\)](#).

2.2.3. Políticas de cortesía, ética y gobernanza en el rastreo web

El desarrollo de sistemas de rastreo automatizado debe observar rigurosos principios de cortesía y cumplimiento de estándares para no perjudicar la disponibilidad de los servidores objetivo [Ferrara et al. \(2014\)](#):

- **Respeto al Robots Exclusion Protocol (`robots.txt`):** Los crawlers éticos deben consultar e interpretar de forma estricta las directivas Disallow y

Crawl-delay publicadas en el archivo raíz de cada dominio antes de emitir solicitudes automatizadas.

- **Limitación de Tasa y Backoff Exponencial:** El software debe implementar retardos deliberados entre peticiones sucesivas al mismo dominio, incorporando mecanismos de retroceso exponencial (*exponential backoff*) ante códigos de estado HTTP 429 (*Too Many Requests*) o 503 (*Service Unavailable*).
- **Identificación Transparente (*User-Agent*):** Las cabeceras HTTP de las peticiones deben incluir un identificador descriptivo que informe al administrador del sistema sobre la naturaleza académica del agente y proporcione un canal de contacto.

2.3. Análisis de plataformas y alternativas existentes

Con el fin de situar la aportación de **UniHub**, se han analizado las principales plataformas institucionales y privadas de información universitaria disponibles en España:

Tabla 2.1

<i>Comparativa entre plataformas existentes de información universitaria y UniHub</i>					
p3.8cm c c c c c Funcionalidad / Característica RUCT BOE QEDU Portales Univ. UniHub					
Catálogo global (Públicas y Privadas)	Sí	No (disperso)	Sí	No (local)	Sí
Desglose curricular de asignaturas (BOE)	No	Sí (PDF plano)	No	Sí (heterogéneo)	Sí (estructurado)
Detección de Menciones oficiales	No	No	No	Parcial	Sí (verificado)
Tratamiento diferenciado de Doctorados	Parcial	Parcial	No	Parcial	Sí (RD 99/2011)
Búsqueda por proximidad (Haversine)	No	No	No	No	Sí (50+ ciudades)
Simulador financiero de matrícula	No	No	No	No	Sí (Púb. y Priv.)
API REST documentada y pública	No	Parcial	No	Raras excepciones	Sí (FastAPI/Swagger)
Arquitectura contenerizada (Docker)	No pública	No pública	No pública	Propietaria	Sí (Docker Compose)

A continuación se detalla el comportamiento de cada alternativa:

- **Registro de Universidades, Centros y Títulos (RUCT - Ministerio):** Constituye el inventario administrativo oficial. Aunque su cobertura es exhaustiva para verificar la vigencia legal de centros y títulos, su interfaz está diseñada con paradigmas heredados de principios de los años 2000. No desglosa las asignaturas aprobadas, no enlaza directamente a los planes publicados en el BOE y carece de un servicio web moderno para su consulta programática por terceros.
- **Buscador del Boletín Oficial del Estado (BOE):** Almacena las resoluciones normativas de publicación de planes de estudio. No obstante, se limita a ofrecer documentos PDF escaneados o vectoriales organizados cronológicamente, sin vincularlos a metadatos relacionales ni comprobar la vigencia frente a planes extinguidos por modificaciones curriculares posteriores.

- **Qué Estudiar y Dónde en la Universidad (QEDU):** Aplicación del Ministerio orientada a la toma de decisiones vocacionales. Proporciona datos de notas de corte y rendimiento de inserción laboral, pero omite por completo los planes de estudios detallados, los temarios docentes y el cálculo dinámico de costes de matrícula.
- **Portales Propietarios de las Universidades:** Presentan el desglose de sus propios grados, pero varían drásticamente en calidad y accesibilidad: mientras algunas universidades disponen de portales interactivos avanzados, otras publican fichas estáticas o documentos PDF escaneados, haciendo imposible la comparativa ágil entre titulaciones de distintas instituciones.

2.4. Aportación y valor diferencial de UniHub

El sistema **UniHub** supera las limitaciones descritas integrando los datos oficiales en un ciclo de vida completo de ingeniería:

1. **Ingesta y Normalización Automatizada:** Un motor híbrido en Python que descarga la base del RUCT, sigue la trazabilidad hasta los planes de estudio aprobados en el BOE, analiza las tablas curriculares mediante segmentación geométrica y rescata asignaturas de los portales web universitarios ante resoluciones sintéticas.
2. **Validación Matemática de Calidad de Datos:** En consonancia con las metodologías de evaluación de calidad de datos propuestas por Batini et al. [Batini et al. \(2009\)](#), el sistema implementa la regla de cierre curricular $\sum \text{ECTS} \geq \text{ECTS}_{\text{exigidos}}$ para clasificar automáticamente cada plan en verificado, parcial o pendiente.
3. **Integración Completa del Marco Legal de Doctorados:** Diseña un modelo de persistencia específico (`programa_doctoral JSONB`) que almacena y visualiza las líneas de investigación acreditadas, actividades formativas y escuelas de doctorado conforme al RD 99/2011 [Ministerio de Educación \(2011\)](#), integrando el concepto de tutela académica anual.
4. **Transparencia Económica y Simulador Financiero:** Codifica las tablas de precios públicos de las 17 comunidades autónomas y honorarios de universidades privadas, permitiendo al usuario calcular con exactitud la liquidación estimada considerando repeticiones y exenciones sociales.
5. **Experiencia de Usuario Moderna y Geolocalizada:** SPA responsiva en React basada en la identidad visual de la Universidad de Cádiz (UCA), con ordenación por proximidad mediante la fórmula del semiverseno (Haversine) y un panel de administración con telemetría de infraestructura en tiempo real.

3. Plan de gestión de proyecto

En esta sección se describen todos los aspectos relativos a la gestión del proyecto **UniHub**: metodología, organización, desglose de costes, planificación temporal con diagrama de Gantt, gestión de riesgos y aseguramiento de la calidad.

3.1. Metodología de desarrollo

Se ha adoptado una metodología de desarrollo ágil e incremental estructurada en fases bien delimitadas. Cada fase se concibe como un subsistema desacoplado con responsabilidades funcionales claras, validado mediante pruebas unitarias e integrales antes de consolidar los artefactos:

- **Fase 0 - Análisis, Requisitos y Estado del Arte:** Estudio normativo (RD 822/2021, RD 99/2011), análisis de viabilidad técnica, arquitectura de datos y revisión bibliográfica.
- **Fase 1 - Motor de Rastreo, Ingesta y Extracción Curricular:** Desarrollo del pipeline de descarga del RUCT, filtrado de activas, deduplicación de planes, parser de resoluciones PDF del BOE, crawling web de apoyo, catálogo de precios SIIU y extracción de guías docentes.
- **Fase 2 - Almacenamiento Relacional y API REST:** Diseño del esquema DDL en PostgreSQL, rol de solo lectura de la API, pipeline de carga masiva ETL con control de sobrescritura, desarrollo de la API REST en FastAPI y telemetría de rendimiento de contenedores.
- **Fase 3 - Portal Web SPA y Simulador Financiero:** Interfaz reactiva en React 19 con diseño institucional UCA, catálogo interactivo con paginación optimizada, modal de planes de estudio y menciones, tarjeta dedicada de doctorados, motor de simulación de matrícula con exenciones y panel de administración aislado.
- **Fase 4 - Contenerización, Orquestación y Pruebas:** Creación de Dockerfiles multi-stage, orquestación mediante Docker Compose con persistencia en volúmenes, batería de más de 400 pruebas automatizadas y despliegue final.
- **Fase 5 - Documentación y Memoria Técnica:** Redacción de la memoria del Trabajo Fin de Grado, elaboración de diagramas UML formales y redacción de manuales de usuario y de desarrollo.

3.2. Tecnologías empleadas

Las tecnologías y herramientas utilizadas se resumen a continuación:

- **Fase 1 (Crawler / ETL):** Python 3.12, xlrd (lectura de XLS BIFF8 del RUCT), BeautifulSoup4 (análisis HTML semántico), pdfplumber y pypdfium2 (segmentación geométrica de tablas PDF en BOE), playwright (navegación headless para portales SPA dinámicos), psutil (medición de telemetría).
- **Fase 2 (API & Base de Datos):** PostgreSQL 15, SQLAlchemy 2.0 (ORM), FastAPI (framework web asíncrono), Pydantic v2 (validación estricta de esquemas de datos), Uvicorn (servidor ASGI) y psycpg2-binary.
- **Fase 3 (Portal Web):** Node.js 20, Vite 8, React 19 (JSX), CSS moderno con variables institucionales UCA, Lucide Icons, Geolocation API (fórmula de Haversine) y Oxlint (análisis estático).
- **Fase 4 (Despliegue y DevOps):** Docker Engine, Docker Compose v2, Nginx 1.25-alpine e imágenes base Debian slim de Python 3.12.
- **Herramientas de soporte:** Visual Studio Code, Git, GitHub y MiKTeX con pdflatex.

3.3. Planificación temporal y diagrama de Gantt

Conforme a la normativa académica de la Universidad de Cádiz para el Trabajo Fin de Grado en Ingeniería Informática, la asignatura cuenta con una asignación de 18 créditos ECTS. Considerando una equivalencia reglamentaria de 25 horas de dedicación por cada crédito ECTS, el proyecto contempla una carga total de trabajo de **450 horas**.

Estas 450 horas de ingeniería se han distribuido a lo largo de un período de 16 semanas laborables, estructuradas en las fases descritas en la Tabla 3.1.

Tabla 3.1

Distribución de carga de trabajo por fases del proyecto (Total: 450 horas)

Fase	Denominación y Actividades Principales	Horas	Porcentaje
Fase 0	Análisis previo, estado del arte, especificación de requisitos	35 h	7,8 %
Fase 1	Motor de rastreo, ingesta RUCT, parsing BOE, web y precios	135 h	30,0 %
Fase 2	Diseño físico PostgreSQL, pipeline ETL y API REST FastAPI	80 h	17,8 %
Fase 3	Desarrollo de frontend React SPA, modal de planes y simulador	110 h	24,4 %
Fase 4	Dockerización, orquestación, persistencia y test suite	50 h	11,1 %
Fase 5	Redacción de memoria técnica, diagramación UML y revisión	40 h	8,9 %
Total	Dedicación total del Trabajo Fin de Grado (18 ECTS × 25 h)	450 h	100,0 %

En la Figura 3.1 se esquematiza el cronograma temporal del proyecto mediante un diagrama de Gantt, ilustrando las dependencias entre hitos y las semanas de ejecución.

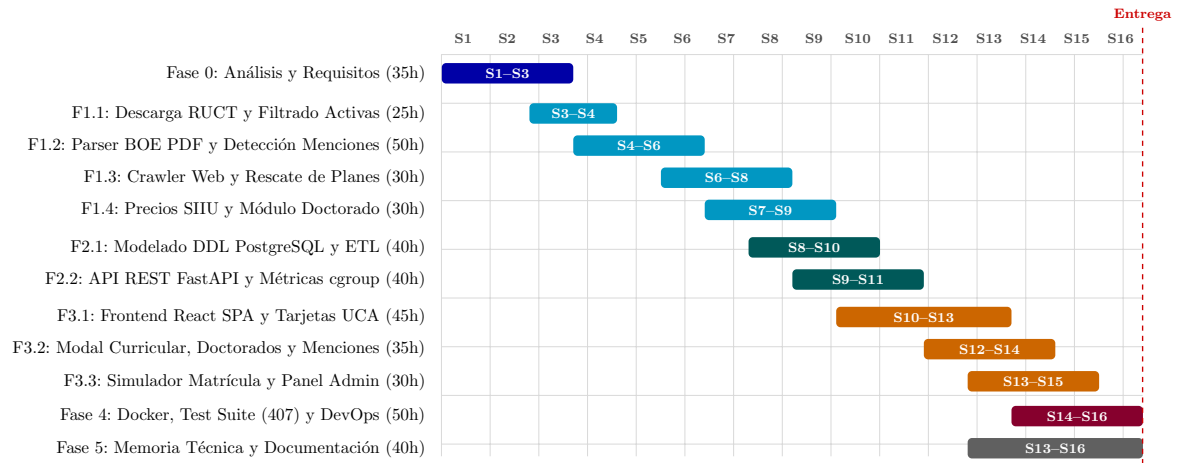


Figura 3.1: Diagrama de Gantt del proyecto UniHub (Planificación de 450 horas en 16 semanas)

3.4. Presupuesto y desglose de costes

El análisis económico del proyecto contempla los costes directos de recursos humanos, equipamiento hardware, software y suministros indirectos, conforme a las tarifas habituales en ingeniería de software. Las horas de dedicación reflejan fielmente el esfuerzo requerido de **450 horas** ($18 \text{ créditos ECTS} \times 25 \text{ h/crédito}$).

En la Tabla 3.2 se detallan todos los conceptos presupuestarios del proyecto.

3.5. Gestión de riesgos

Se identificaron y mitigaron los siguientes riesgos principales:

- **R-01: Modificaciones en las plantillas web o cortes de red.** Impacto: Alto. Mitigación: Parsers dinámicos con captura de excepciones, registro forense en `errores_crawler.json` y mecanismos de cortocircuito ante caídas prolongadas.
- **R-02: Sobrescritura accidental de datos completos con registros parciales.** Impacto: Crítico. Mitigación: Verificación obligatoria de completitud curricular ($\sum \text{ECTS} \geq \text{ECTS}_{\text{exigidos}}$) antes de actualizar la persistencia en base de datos.
- **R-03: Pérdida de persistencia al reiniciar o apagar contenedores Docker.** Impacto: Crítico. Mitigación: Asignación de volúmenes nominales persistentes (`unihub_postgres_data`) y bind mounts hacia el host.
- **R-04: Degradación del rendimiento de la API ante consultas concurrentes.** Impacto: Medio. Mitigación: Creación de índices B-Tree en columnas clave (`codigo_estudio`, `universidad_codigo`), rol de solo lectura y agrupaciones pre-agregadas en la base de datos.

Tabla 3.2

Presupuesto detallado del proyecto UniHub (450 horas de trabajo)

p6.8cm r r r **Concepto / Recurso Unidades / Horas Precio Unitario Coste Total**

Fase 0: Análisis y Requisitos	35 h	30,00 €/h	1.050,00 €
Fase 1: Rastreador Python y Extracción BOE	135 h	30,00 €/h	4.050,00 €
Fase 2: Base de Datos PostgreSQL y API FastAPI	80 h	30,00 €/h	2.400,00 €
Fase 3: Portal Web React SPA y Calculadora	110 h	30,00 €/h	3.300,00 €
Fase 4: Docker, Persistencia y Batería de Tests	50 h	30,00 €/h	1.500,00 €
Fase 5: Memoria Técnica y Documentación	40 h	30,00 €/h	1.200,00 €
Subtotal Personal (450 horas)	450 h	30,00 €/h	13.500,00 €
Estación de trabajo portátil (1.200 €, amort. 4 meses)	4 meses	25,00 €/mes	100,00 €
Dispositivos de prueba y almacenamiento externo	4 meses	7,50 €/mes	30,00 €
Subtotal Equipamiento			130,00 €
Pila FOSS (Python, PostgreSQL, FastAPI, React, Docker)	Licencias Open Source		
		0,00 €	0,00 €
Subtotal Software			0,00 €
Conexión a Internet de banda ancha (4 meses)	4 meses	35,00 €/mes	140,00 €
Consumo eléctrico imputable al desarrollo	4 meses	27,50 €/mes	110,00 €
Subtotal Costes Indirectos			250,00 €
Base Imponible (Coste Total Directo e Indirecto)			13.880,00 €
Impuesto sobre el Valor Añadido (IVA 21 %)			2.914,80 €
PRESUPUESTO TOTAL DEL PROYECTO			16.794,80 €

3.6. Aseguramiento de la calidad y gestión de la configuración

El aseguramiento de la calidad se apoya en una batería integral de 407 pruebas automatizadas que cubren todas las fases del ciclo de vida (crawler, lógica DDL, API REST, simulación de matrícula, geolocalización y contenerización). El control de versiones se gestiona mediante Git bajo un flujo de trabajo de commits atómicos e independientes asociados a funcionalidades concretas.

Parte II

Desarrollo

4. Requisitos del Sistema

En este capítulo se presentan los objetivos del sistema y el catálogo completo de requisitos funcionales y no funcionales de la plataforma **UniHub**.

4.1. Objetivos de Negocio

El objetivo de negocio principal es ofrecer un portal público centralizado de enseñanza superior en España, facilitando la transparencia, la comparativa de planes de estudio BOE y el acceso a la oferta universitaria pública y privada.

4.2. Objetivos del Sistema

1. Automatizar la extracción periódica de universidades y titulaciones vigentes (Grados, Másteres y Doctorados). 2. Homogenizar y almacenar relacionalmente los datos en PostgreSQL ordenando públicamente primero las universidades públicas y posteriormente las privadas. 3. Exponer una API REST documentada en FastAPI con control de permisos y métricas físicas cgroup. 4. Construir un portal web interactivo con estética corporativa de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100) y ocultación de códigos internos. 5. Proporcionar un buscador por geolocalización (cercanía en km con distancia integrada directamente en tarjetas). 6. Recolectar métricas de uso y clasificar las 6 universidades más visitadas en "Universidades Destacadas". 7. Incorporar el apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA). 8. Aislar el panel de control del Administrador de la Web tras la ruta manual /admin. 9. Desplegar la infraestructura mediante contenedores Docker orquestados con persistencia garantizada.

4.3. Requisitos funcionales

- **RF-01 Descarga de Catálogo Oficial y Computación Paralela:** Descargar e inspeccionar catálogos de universidades y titulaciones mediante una arquitectura en 2 procesos paralelos (Productor de red I/O y Consumidor de análisis CPU).
- **RF-02 Filtrado Estricto de Vigencia y Clasificación de Doctorados:** Excluir titulaciones extinguidas, no vigentes o eliminadas; conservar únicamente las vigentes o autorizadas y clasificar y extraer los programas de Doctorado (RD 99/2011) en base a sus líneas de investigación tutelada, escuela doctoral y actividades formativas sin forzar créditos ECTS lectivos.

- **RF-03 Parseo Meticuloso Multi-PDF de BOE:** Escanear todos los PDF candidatos del BOE para extraer y fusionar asignaturas, módulos, materias, créditos ECTS, curso y cuatrimestre sin detenerse en la primera coincidencia.
- **RF-04 Resiliencia y Notificación Automática:** Aplicar pausas de 5 min (10 fallos) y cortocircuito a los 15 min ante caídas de red, notificando via POST a la API REST al finalizar.
- **RF-05 Regla de No Sobrescritura Vacía:** Evitar que campos vacíos o indeterminados (, undefined") sobrescriban información previa válida.
- **RF-06 API REST de Consulta con Ordenación Prioritaria:** Endpoints para listar universidades (públicas primero, privadas después) y titulaciones por nivel académico.
- **RF-07 API REST CRUD de Administración y Métricas cgroup:** Endpoints POST, PUT y DELETE para universidades y titulaciones, junto con métricas de uso de RAM/CPU por contenedor.
- **RF-08 Buscador, Filtros y Paginación Web:** Búsqueda global por nombre, tipo (Pública/Privada), CCAA y nivel (Grado, Máster, Doctorado), con paginación de 5, 10, 20, 50 y 100 resultados.
- **RF-09 Ocultación de Códigos Internos:** Ocultar códigos internos numéricos en la interfaz pública web.
- **RF-10 Visor Modal de Planes BOE:** Visualización en la web del desglose ECTS y enlace oficial al PDF del BOE.
- **RF-11 Búsqueda por Cercanía con Distancia Integrada:** Cálculo de distancias en km mediante la fórmula de Haversine mostrando la distancia dentro de la tarjeta del centro.
- **RF-12 Universidades Destacadas (Top 6):** Cálculo dinámico de las 6 universidades más consultadas en la plataforma.
- **RF-13 Sección "Sobre Nosotros":** Página informativa del TFG de Alejandro Ramos Rodríguez (Ingeniería Informática, UCA).
- **RF-14 Panel del Administrador en Ruta Manual /admin:** Acceso aislado sin botón visible para gestionar CRUD y monitorear la salud individual de contenedores.
- **RF-15 Rastreo Paralelo de Webs Oficiales (Fase 1 - Parte 2):** Escanear las webs oficiales de las universidades para titulaciones sin plan de estudios o incompletas, verificando estrictamente robots.txt, analizando el Sitemap XML, resolviendo widgets dinámicos asíncronos mediante atributos HTML5 (data-config, data-url) y llamadas a servicios REST institucionales, buscando mediante sinónimos académicos y guardando la URL directa de acceso en web_fuente_directa_url.
- **RF-16 Validación Matemática de Completitud Curricular:** Validar que el volumen total de créditos ECTS obtenidos en las asignaturas sea $\geq$ a los

créditos oficiales exigidos por ley (Grado estándar 240, Medicina 360, Odontología/Farmacología/Veterinaria/Arquitectura 300, Máster 120/90/60). Para Doctorados regulados por el RD 99/2011, exigir 0 créditos ECTS lectivos y validar estructuralmente la presencia de líneas de investigación científica y Escuela de Doctorado. Si un plan formativo de Grado o Máster es incompleto o solo resumen, activar automáticamente el rastreo en la web oficial.

- **RF-17 Priorización Contextual Semántica de Enlaces en RUCT:** Clasificar los enlaces a PDFs del BOE según el fieldset contenedor en la ficha del RUCT, dando máxima prioridad a los enlaces de *Publicación Plan de Estudios* y *Fechas de Corrección Plan Estudio* (prioridad 90–100) y relegando los *Acuerdos de Consejo de Ministros* (prioridad 10).

4.4. Requisitos no funcionales

- **RNF-01 Rendimiento:** Latencia promedio de respuesta de la API REST inferior a 100 ms.
- **RNF-02 Usabilidad e Identidad Visual:** Interfaz web profesional con paleta corporativa de la Universidad de Cádiz (UCA).
- **RNF-03 Seguridad:** Rol de solo lectura en base de datos (`unihub_api_user`) y acceso restringido por ruta manual al panel de administración.
- **RNF-04 Resiliencia a Fallos:** El crawler no se detendrá ante errores individuales de red o parseo de PDFs, registrándolos en `errores_crawler.json` y aplicando el cortocircuito de 5m/15m.
- **RNF-05 Persistencia y Portabilidad:** Despliegue en 4 contenedores Docker independientes con persistencia garantizada en volumen nominado y montaje directo de host.

4.5. Requisitos de información

El sistema gestiona las siguientes entidades principales de información: *Universidad*, *Titulación*, *PlanEstudios*, *ResumenCréditos*, *ElementoCurricular*, *ErrorCrawler* y *EstadísticaRendimiento*.

4.6. Reglas de negocio

- **RN-01 Jerarquía por Real Decreto:** En presencia de múltiples planes de una misma titulación, priorizar RD 822/2021 ¿ RD 1393/2007 ¿ RD 56/2005.
- **RN-02 No Omisión Entre Universidades:** Misma titulación en distintas universidades debe ser tratada como titulación independiente.

- **RN-03 Comprobación Incremental Estricta:** No volver a descargar un PDF del BOE si la URL y la fecha coinciden con el registro de `checkpoint.json`.
- **RN-04 Ordenación Prioritaria:** Mostrar siempre las universidades públicas en primer lugar y posteriormente las privadas.
- **RN-05 Cero Valores por Defecto Arbitrarios:** Prohibir cualquier estimación artificial o fallback estático de tarifas o créditos; todo dato debe provenir estrictamente de las fuentes oficiales (BOE, RUCT, SIIU, web oficial).

4.7. Análisis GAP

Dado que **UniHub** se ha construido a medida mediante una arquitectura en 4 fases que integra rastreo, API REST propia, frontend con diseño UCA y orquestación Docker, no se requirió la adopción directa de plataformas base de terceros.

5. Análisis del Sistema

Este capítulo cubre el análisis conceptual, funcional y dinámico del sistema de información **UniHub**, haciendo uso del lenguaje de modelado UML y modelos de comportamiento formalizados.

5.1. Modelo Conceptual y de Dominio

El modelo conceptual identifica las entidades centrales del dominio universitario y sus asociaciones estructurales:

- **Universidad:** Representa cada centro universitario público o privado registrado en el RUCT (código oficial, nombre, tipo de centro, comunidad autónoma, municipio, provincia, URL web oficial, email de contacto y teléfono institucional).
- **Titulacion:** Representa una titulación oficial vigente de Grado, Máster o Doctorado (código de estudio, denominación oficial, nivel académico, estado de vigencia y rama de conocimiento). Se relaciona mediante agregación con Universidad (1:N).
- **PlanEstudios:** Contiene la metainformación del plan de estudios (URL del BOE, fecha de publicación, fecha de procesado, procedencia de la fuente, tipo de estructura curricular, estado de calidad verificado, diagnóstico de completitud matemática y objeto estructurado `programa_doctoral` para Doctorados). Se relaciona 1:1 con Titulacion.
- **ResumenCreditos:** Almacena la distribución oficial de créditos ECTS (formación básica, obligatoria, optativa, TFG/TFM, prácticas externas y créditos totales exigidos). Relación N:1 con PlanEstudios.
- **ElementoCurricular:** Almacena el desglose estructurado de asignaturas o líneas de investigación científica (carácter *INVESTIGACION*), módulos, materias, créditos ECTS, curso y cuatrimestre. Relación N:1 con PlanEstudios.
- **TarifaECTS:** Almacena la estructura de precios públicos por crédito ECTS (1^a a 4^a matrícula) e importes estimados anuales procedentes del SIIU y decretos autonómicos. Relación N:1 con Titulacion.

5.2. Modelo de Casos de Uso

Se identifican tres actores que interactúan con el sistema según su perfil de permisos:

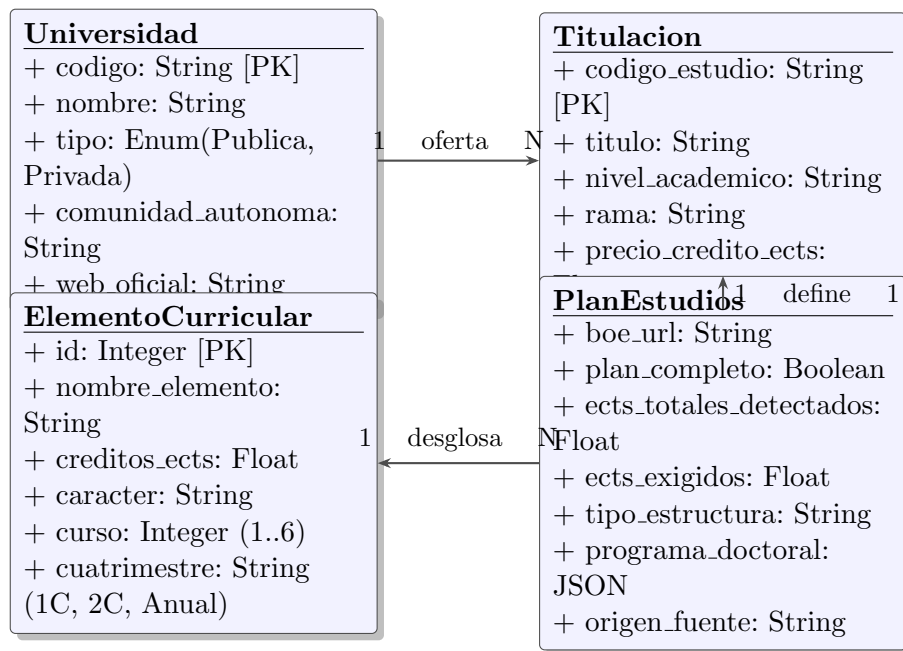


Figura 5.1: Diagrama de Clases del Modelo de Dominio de UniHub (UML)

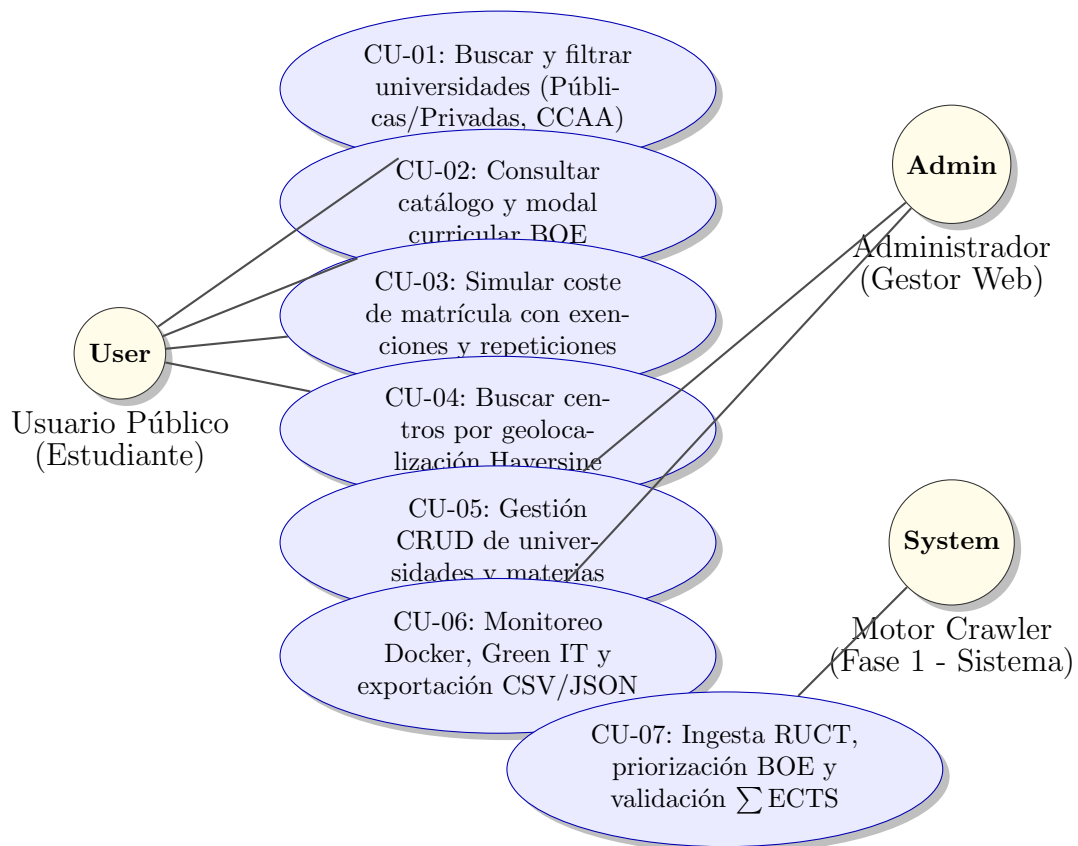


Figura 5.2: Diagrama General de Casos de Uso de UniHub (UML)

5.3. Modelo de Comportamiento Dinámico

El comportamiento del sistema se describe a través de dos modelos de interacción fundamentales:

5.3.1. Modelo de Comportamiento del Rastreador y Validación Curricular (Fase 1)

El flujo de ejecución del rastreador combina la priorización contextual de enlaces HTML en RUCT con la validación matemática de completitud curricular:

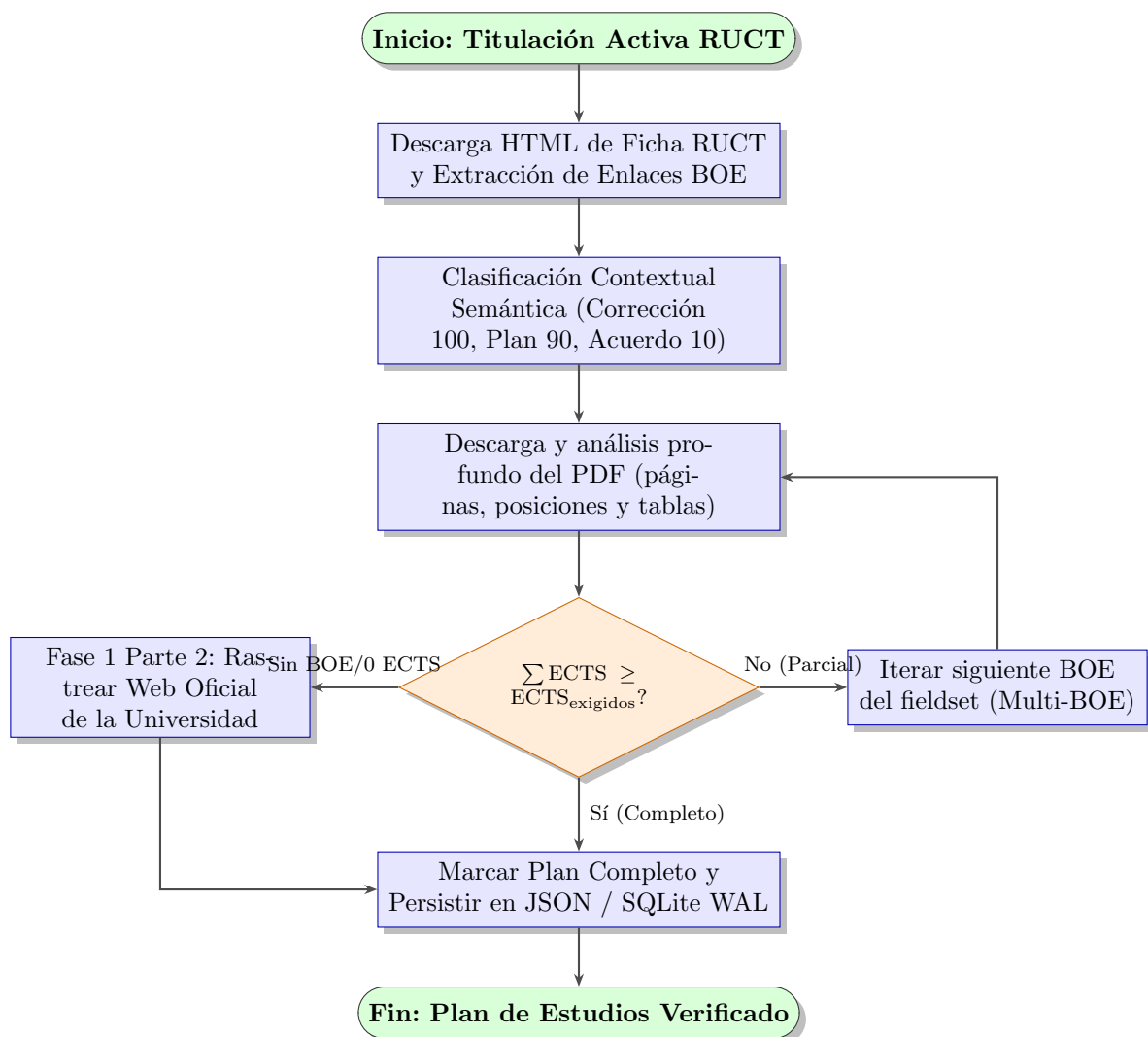


Figura 5.3: Diagrama de Actividad: Ingesta Inteligente y Validación Matemática de Completitud Curricular

5.3.2. Modelo de Secuencia: Simulación Financiera en Calculadora de Matrícula

En la Fase 3, el usuario interactúa dinámicamente con la calculadora de matrícula para simular el coste oficial de liquidación académica. El flujo de interacción y los intercambios de mensajes entre la interfaz de usuario, el servicio de backend y la base de datos se ilustran formalmente en el Diagrama de Secuencia UML de la Figura 5.4.

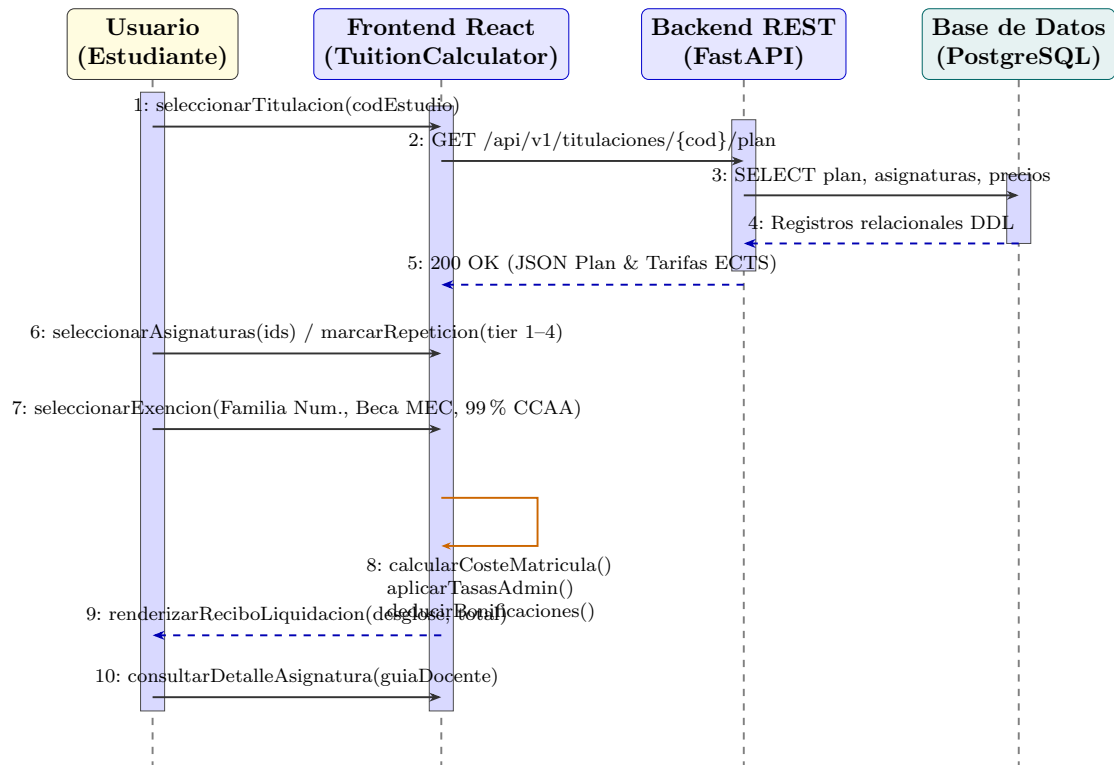


Figura 5.4: Diagrama de Secuencia UML: Interacción en la Simulación Financiera de Matrícula (Fase 3)

El flujo operativo formal se desarrolla a través de las siguientes etapas interconectadas:

- Recuperación de datos oficiales:** Al seleccionar una titulación en el desplegable, la aplicación React emite una solicitud asíncrona `GET` a la API REST de FastAPI para recuperar el plan de estudios verificado y las tarifas oficiales por crédito ECTS (1ª, 2ª, 3ª y 4ª+ matrícula). La API ejecuta la consulta contra PostgreSQL y devuelve el objeto JSON estructurado.
- Selección curricular y ponderación de repetición:** El usuario activa o desactiva asignaturas individuales o cursos completos. Por cada asignatura seleccionada, puede indicar el grado de repetición (1,0×, 1,5×, 3,0× o 4,5× sobre el precio del crédito).
- Aplicación de exenciones y bonificaciones sociales:** El estudiante puede activar regímenes especiales: Familia Numerosa General (50 %), Especial (100 %), Beca MEC (exención de 1ª matrícula), Matrícula de Honor (exención de hasta

60 ECTS) o Bonificación del 99 % de la Junta de Andalucía para asignaturas aprobadas en primera convocatoria.

4. **Cálculo reactivo en tiempo de ejecución:** La función memoizada en React (`useMemo`) computa el importe total de créditos, las tasas administrativas de secretaría y el descuento aplicado en un tiempo inferior a 1 ms, sin requerir peticiones de red adicionales y actualizando instantáneamente el recibo de liquidación desglosado.

5.4. Modelo de Interfaz de Usuario

El diseño de interfaz SPA se estructura en componentes modulares React con soporte nativo de la History API:

- **Barra de Navegación Nivel Superior (Navbar):** Logotipo UniHub, pestañas de navegación (Inicio, Universidades, Titulaciones, Calculadora, Cercanía, Sobre Nosotros) y selector de modo oscuro.
- **Pantalla de Inicio (Hero):** Banner corporativo estilo UCA, métricas globales dinámicas y grilla interactiva de “Universidades Destacadas”.
- **Grilla de Tarjetas con Paginación (UnivCard / DegreeCard):** Vista de fichas con ocultación de códigos internos, distancias en kilómetros integradas y distintivo visual de plan completo verificado.
- **Modal de Plan de Estudios (PlanModal):** Cuadro de diálogo flotante con desenfoque de fondo (*glassmorphism*) que lista la estructura curricular, itinerarios de mención y enlace al PDF oficial del BOE.
- **Panel de Administración Aislado (AdminDashboard):** Vista de acceso seguro vía URL manual `/admin` con monitoreo en tiempo real de contenedores Docker, semáforos Core Web Vitals, telemetría Green IT y tablas CRUD interactivas con exportación CSV/JSON protegida.

6. Diseño del Sistema

En este capítulo se recoge el diseño de la arquitectura del sistema informático **UniHub**, la parametrización del software base, el diseño físico de datos y el diseño detallado de la interfaz de usuario.

6.1. Arquitectura del Sistema

Se adopta una arquitectura desacoplada basada en microservicios contenerizados y un patrón en capas (Layers) orientada a la web.

6.1.1. Diseño de alto nivel

Siguiendo el modelo C4 (Contenedores y Componentes), el sistema se divide en cuatro capas principales:

1. **Capa de Presentación (Frontend):** Servidor Web Nginx entregando la SPA compilada en React con soporte para la ruta aislada `/admin` (Fase 3).
2. **Capa de Servicio (API REST):** Framework FastAPI en Python que expone controladores RESTful, métricas cgroup de contenedores, sincronización reactiva `/api/v1/admin/sync-etl` y endpoints CRUD (Fase 2).
3. **Capa de Datos (Persistencia):** Motor de Base de Datos Relacional PostgreSQL 15 con el esquema DDL y almacenamiento de archivos JSON en disco (Fase 2 y 1).
4. **Capa de Ingesta (Crawler):** Módulo independiente en Python organizado en cuatro partes: catálogo RUCT y resoluciones BOE, recuperación desde webs oficiales, consolidación de precios ECTS y enriquecimiento con guías docentes. La ejecución puede programarse mediante Cron y mantiene un manifiesto y puntos de control para su auditoría.

6.1.2. Diseño detallado

La arquitectura de ingesta y servicios se rige por los siguientes componentes clave:

- **Recuperación desde webs oficiales (Fase 1 Parte 2):** Cuando el BOE no publica el detalle suficiente, el crawler explora de forma acotada las webs institucionales, sus sitemaps y catálogos relacionados. La resolución usa normalización léxica y límites de profundidad configurables, incorporando además el extractor universal de widgets dinámicos HTML5 / endpoints REST y el marco agnóstico de extracción de programas de doctorado (RD 99/2011) con navegación canónica a subpáginas de líneas acreditadas y descarte de binarios.

- **Garantía de Rastreo Ético y Cortesía Monodominio:** La exploración de los catálogos y subpáginas de una misma universidad se ejecuta de forma estrictamente secuencial a través del cliente `RUCTDownloader`, aplicando retardos corteses (`REQUEST_DELAY` y dispersión aleatoria *Jitter*), respetando las directivas `Crawl-delay` del protocolo `robots.txt` (con caché conforme a RFC 9309 con desalojo LRU) y activando el patrón *Circuit Breaker* ante incidencias continuadas de servidor para aislar el dominio conflictivo.
- **Arquitectura de Caché Jerárquica Dual (L1 RAM / L2 SQLite WAL):** El sistema implementa un esquema de almacenamiento intermedio en dos niveles para guías docentes y URLs candidatas. Las peticiones resueltas o descartadas (códigos 404/410) se consolidan en tablas SQLite bajo modo *Write-Ahead Logging* (WAL) y se replican en diccionarios de memoria RAM protegidos por cerrojos reentrantes, permitiendo omitir enlaces no válidos en 0 ms y minimizando el tráfico de red.
- **Caché Criptográfica de Modelos de Documento (SHA-256):** Para optimizar el análisis de resoluciones del BOE que aprueban múltiples titulaciones de forma simultánea, el analizador vectorial mantiene un búfer LRU en memoria indexado por la huella digital SHA-256 del contenido del PDF, garantizando tiempo de análisis de $\approx 0,9$ ms en lecturas secundarias.
- **Capa de Normalización Curricular y Detección Idiomática:** Módulo desacoplado de enriquecimiento que infiere automáticamente la lengua académica de impartición (*Castellano, Català, Galego, Euskara, English*) y normaliza los criterios de calificación en un esquema porcentual estructurado (`teoria_porcentaje`, `practicas_porcentaje`, `evaluacion_continua_porcentaje`, `examen_final_porcentaje`).
- **Centralización y Parametrización en `config.py`:** Todos los parámetros y cotas del sistema de ingesta (`HUB_AND_SPOKE_MAX_HUBS`, `HUB_AND_SPOKE_MAX_DEPTH`, `HUB_ACADEMIC_KEYWORDS`, `timeouts`, `reintentos` y cuotas ECTS) se encuentran formalmente centralizados en el archivo de configuración con soporte para sobreescritura dinámica mediante variables de entorno del sistema operativo analizadas con convertidores protegidos contra excepciones.
- **`routes/universidades.py`:** Endpoints con ordenación prioritaria (Públicas primero) y operaciones CRUD.
- **`routes/titulaciones.py`:** Endpoints para Grados, Másteres y Doctorados, lectura de planes de estudio BOE, filtrado vocacional por Rama de Conocimiento (`rama`) y controladores CRUD completos para la gestión de asignaturas y elementos curriculares (`GET/POST /api/v1/titulaciones/{codigo}/asignaturas`, `PUT/DELETE /api/v1/titulaciones/asignaturas/{id}`).
- **`routes/estadisticas.py`:** Endpoint `/api/v1/estadisticas/cobertura` que expone métricas calculadas sobre los datos cargados. Los porcentajes de cobertura, caché o huella deben interpretarse junto con la fecha, el manifiesto y la configuración de la ejecución de origen.
- **`metrics/container_metrics.py`:** Muestreo de métricas físicas de salud (RAM RSS MB, CPU %) y huella de carbono (gCO_2) por contenedor cgroup.

6.1.3. Seguridad y Control de Acceso (RBAC)

La API REST implementa un modelo de Control de Acceso Basado en Roles (RBAC) para proteger las operaciones críticas y la integridad de los datos de producción:

- **Usuarios Estándar (Lectura):** El 99 % de los consumidores (incluyendo el frontend público) interactúan a través de endpoints GET sin autenticación, respaldados en base de datos por un usuario `unihub_api_user` con privilegios estrictos de `GRANT SELECT`.
- **Administradores (Escritura):** Operaciones CRUD (Creación, Modificación, Eliminación de universidades, titulaciones y asignaturas) y ejecución de volcados de datos (ETL) requieren autenticación mediante la cabecera HTTP `X-API-Key`. La verificación utiliza `secrets.compare_digest` en `security.py` para garantizar tiempo constante en la comparación y prevenir ataques de canal lateral (timing attacks).
- **Blindaje de Conexiones a Base de Datos:** En `config.py`, la construcción de la cadena de conexión SQLAlchemy empaqueta los parámetros con `urllib.parse.quote_plus`, desinfectando caracteres especiales en contraseñas o nombres de usuario (`@`, `/`, `%`, etc.) y previniendo inyecciones de credenciales.

6.2. Parametrización del software base

Se configuró el servidor proxy inverso Nginx (`nginx.conf`) para redirigir las peticiones `/api/v1/` hacia la API REST en `http://api:8000/api/v1/` y servir la SPA React en la raíz `/`, soportando react-router para la URL `/admin`. En PostgreSQL, se parametrizó la creación del usuario restringido `unihub_api_user` con permisos `GRANT SELECT` exclusivo, utilizando cadenas de conexión URL-encoded mediante `quote_plus` en el módulo de configuración de la API (`config.py`).

6.3. Diseño Físico de Datos

El diseño relacional en PostgreSQL (`schema.sql`) se compone de las siguientes tablas principales con claves primarias y foráneas con borrado en cascada, optimizadas mediante la extensión de trigramas `pg_trgm`:

- **universidades:** Clave primaria `codigo` `VARCHAR(10)`. Índices en `tipo`, `comunidad_autonoma` e índice GIN trigrama `idx_univ_nombre_trgm` sobre `nombre`.
- **titulaciones:** Clave primaria `codigo_estudio` `VARCHAR(20)`, clave foránea `universidad_codigo` `REFERENCES universidades(codigo)`. Índice GIN trigrama `idx_tit_titulo_trgm` sobre `titulo`.
- **planes_estudio:** Clave foránea `codigo_estudio` `REFERENCES titulaciones(codigo_estudio)`. Incorpora las columnas de trazabilidad `origen_fuente` `VARCHAR(100)` y `pdf_sha256`.

VARCHAR(64), campos de calidad y verificación (`estado_calidad`, `motivos_calidad`, `fuelle_verificada_url`, `verificado_en`), tipo estructural (`tipo_estructura`, `ects_exigidos`) y la columna semiestructurada `programa_doctoral` JSONB para almacenar las líneas de investigación acreditadas, escuela tutelar y actividades formativas bajo el RD 99/2011.

- `resumen_creditos`: Clave foránea a `planes_estudio(id)`.
- `elementos_curriculares`: Clave foránea a `planes_estudio(id)`. Tipos de datos en texto extenso (TEXT) para evitar truncamientos.
- `errores_crawler` y `estadisticas_rendimiento`: Tablas relacionales de registro de auditoría con deduplicación por timestamp.

6.4. Diseño de la Interfaz de Usuario

El diseño visual y la experiencia de usuario (UX/UI) de la aplicación web de **UniHub** se han concebido bajo los estándares modernos de desarrollo de Single Page Applications (SPA) con React 19 y Vite, siguiendo estrictamente la identidad visual corporativa de la **Universidad de Cádiz (UCA)**:

- **Azul Noche UCA (Color Primario y Navegación)**: #002B49 y #003366, empleado en barras de cabecera, tipografía de títulos y banners principales.
- **Azul Mar de Cádiz (Acentos y Acciones)**: #0084C8 y #00A8E8, utilizado en botones de acción principal, enlaces interactivos y bordes destacados.
- **Dorado Sol de Cádiz (Avisos e Insignias Especiales)**: #F3A712 y #FFC72C, aplicado en insignias de advertencia y registros gestionados por administración.
- **Verde Aprobado / Éxito (Insignia Verificado)**: #10B981, aplicado en distintivos de planes completos verificados conforme a publicación en BOE.
- **Rosa Magenta (Distintivo Doctorado)**: #EC4899 (con fondo al 15 % de opacidad), específico para programas oficiales de doctorado e investigación (RD 99/2011).

6.4.1. Vistas Principales y Prototipado del Sistema

A continuación se describen y documentan visualmente las pantallas nucleares de la plataforma mediante capturas de la interfaz implementada.

Pantalla Principal y Catálogo de Universidades

La vista principal integra la barra de navegación superior, el banner institucional con métricas globales en tiempo real y la grilla interactiva de universidades (Figura 6.1).

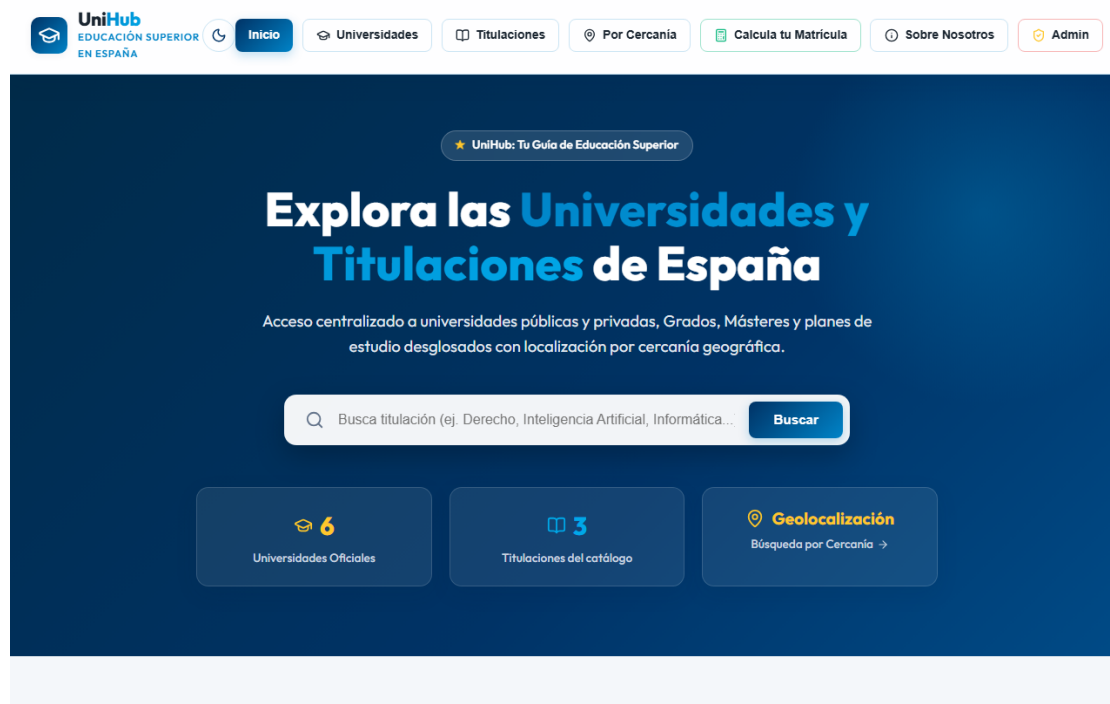


Figura 6.1: Interfaz de usuario: Pantalla de inicio y catálogo interactivo de universidades con buscador y filtros (Fase 3)

Cada tarjeta universitaria expone su denominación oficial, tipología institucional (Pública o Privada), comunidad autónoma y el desglose de oferta académica (Grados, Másteres y Doctorados). Asimismo, incorpora controles de paginación configurable (5, 10, 20, 50 y 100 elementos por página) con desplazamiento suave automático al inicio tras cada transición.

Modal de Visualización Curricular y Menciones Oficiales

Al pulsar sobre el botón de consulta de una titulación, el sistema despliega un diálogo modal flotante con desenfoque de fondo (*glassmorphism*) que renderiza la estructura docente (Figura 6.2).

El modal incluye:

1. **Encabezado de Calidad y Trazabilidad:** Muestra la insignia de calidad del plan (*Verificado BOE, Universidad o Administración*) y proporciona el botón directo hacia la disposición legal original.
2. **Desglose Curricular por Cursos y Cuatrimestres:** Agrupa las asignaturas estructuradas (1º a 4º curso; 1C, 2C y Anual) con indicación de tipología (Básica, Obligatoria, Optativa) y carga en créditos ECTS.
3. **Menciones Curriculares e Itinerarios:** Detección y filtrado visual de itinerarios formativos oficiales aprobados en el plan de estudios.
4. **Tratamiento Específico de Doctorados:** Para programas de doctorado, susti-

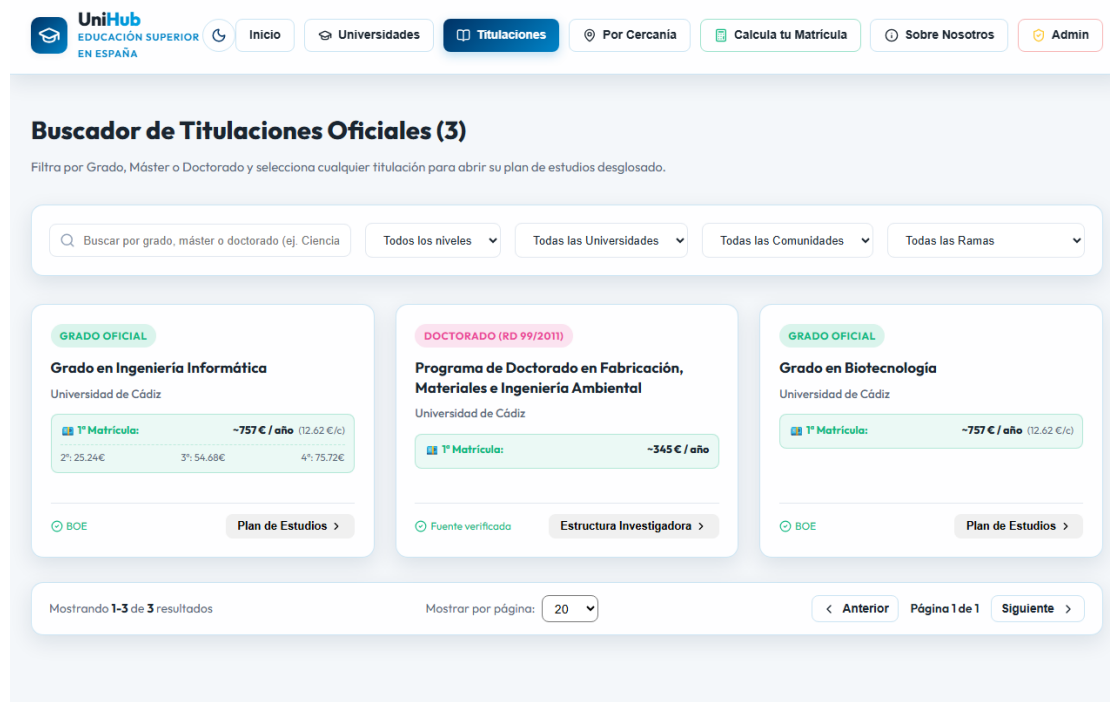


Figura 6.2: Interfaz de usuario: Modal de detalle curricular con asignaturas, créditos ECTS, cuatrimestres y trazabilidad BOE

tuye la tabla de asignaturas por la tarjeta de investigación con líneas acreditadas, escuela tutelar y régimen de tutela académica anual.

Simulador Financiero de Matrícula Universitaria

La herramienta de liquidación de matrícula (*TuitionCalculator*) permite al estudiante estimar con exactitud el coste económico de sus estudios en centros públicos y privados (Figura 6.3).

El simulador implementa en tiempo real:

- Selección granular de asignaturas individuales o cursos completos.
- Modificación del grado de repetición (1^a, 2^a, 3^a o 4^a+ matrícula) aplicando los coeficientes multiplicadores autonómicos.
- Aplicación de exenciones sociales (Familia Numerosa General y Especial, Discapacidad $\geq 33\%$, Beca MEC del Ministerio, Matrícula de Honor y Bonificación del 99 % de la Junta de Andalucía).
- Generación instantánea del recibo de liquidación con desglose de créditos, tasas de secretaría y bonificación total.

Figura 6.3: Interfaz de usuario: Estimador interactivo de liquidación de matrícula con repeticiones y exenciones sociales

Búsqueda por Proximidad Geográfica (Fórmula de Haversine)

La vista de geolocalización permite a los usuarios ordenar el catálogo de centros universitarios en función de su distancia física real en kilómetros (Figura 6.4).

El cálculo se ejecuta en el cliente mediante la fórmula trigonométrica del semi-verseno (Haversine), tomando como referencia las coordenadas GPS del navegador del usuario o una capital de provincia seleccionada manualmente entre las 52 provincias españolas.

6.4.2. Panel de Control y Administración Aislado

Ubicado bajo la ruta de acceso manual `/admin` y protegido mediante autenticación por clave API (`X-API-Key`), el panel de administración centraliza la monitorización y gestión de la plataforma:

1. **Herramientas de Exportación Masiva CRUD:** Descarga en formatos CSV (con codificación UTF-8 BOM para compatibilidad total con hojas de cálculo) y JSON estructurado, sanitizando caracteres iniciales potencialmente peligrosos (`=`, `+`, `-`, `@`) para neutralizar ataques de CSV Injection.
2. **Gestor Modal de Asignaturas y Precios ECTS:** Subpanel para añadir, editar o dar de baja materias curriculares, bloqueando modificaciones accidentales sobre planes verificados mediante la marca `gestionado_por_admin`.

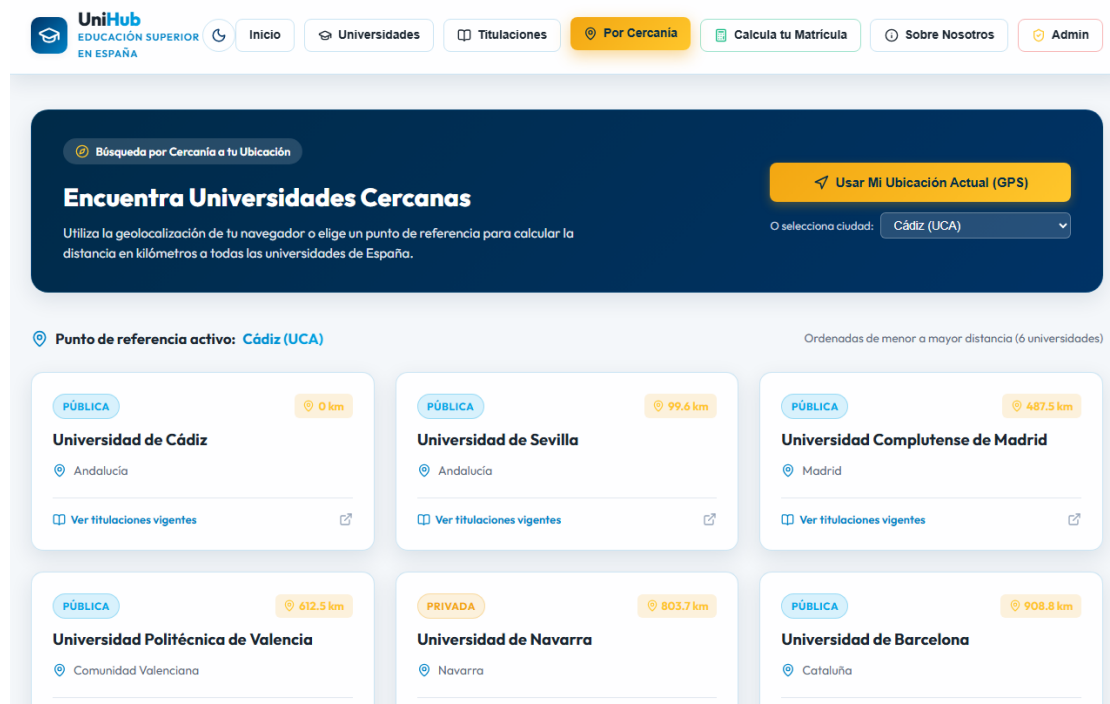


Figura 6.4: Interfaz de usuario: Búsqueda y ordenación de universidades por proximidad geográfica mediante geolocalización

3. **Semáforos Core Web Vitals y Latencia de Base de Datos:** Indicadores de rendimiento en vivo (TTFB, FCP y LCP) conforme a las directrices de Google CWV, complementados con la medición de latencia en milisegundos de la conexión PostgreSQL.
4. **Telemetría Docker y Sostenibilidad Green IT:** Monitorización continua de consumo de memoria RAM RSS y porcentaje de uso de CPU por contenedor, complementado con la estimación de emisiones de carbono (gCO_2).
5. **Monitor de Sincronización ETL en Vivo:** Banner reactivo que notifica el progreso de la carga de datos masiva y la liberación de cerrojos de proceso (etl_running.lock).

7. Construcción del Sistema

Este capítulo trata sobre todos los aspectos relacionados con la implementación en código del sistema **UniHub**, desglosando la arquitectura técnica de cada una de sus cuatro fases de desarrollo.

7.1. Entorno de Construcción

El marco tecnológico de construcción incluye:

- **Entorno de Desarrollo (IDE):** Visual Studio Code / Antigravity Agent.
- **Lenguajes:** Python 3.12, JavaScript (ES6+ / JSX), SQL (PostgreSQL), HTML5, CSS3.
- **Librerías y Módulos Python:** FastAPI, Uvicorn, SQLAlchemy, Pydantic v2, httpx[http2], h2, requests, psycpg2-binary, xlrd, BeautifulSoup4, pdfplumber, pypdf, psutil, multiprocessing, concurrent.futures, urllib.robotparser.
- **Librerías y Herramientas JavaScript:** React 19, Vite 8, Oxlint, Lucide React (iconografía UCA), usageTracker analytics, perfTracker performance telemetry.
- **Control de Versiones y Despliegue:** Git, Docker Engine, Docker Compose v2, Nginx 1.25-alpine.

7.2. Código Fuente y Estructura Modular

El código fuente del proyecto se organiza de forma modular bajo el directorio `Codigo/`:

7.2.1. Fase 1: Módulo de Ingesta y Crawler (`Codigo/Crawler/`)

El sistema de ingesta de la Fase 1 se estructura en cuatro partes desacopladas y especializadas:

Fase 1 - Parte 1: Ingesta del RUCT y Parser PDF del BOE

- **config.py:** Módulo centralizado de configuración estructurado en 10 secciones temáticas claramente documentadas (Rutas de Archivos, Persistencia Dual, Endpoints Ministeriales del RUCT, Red y Pooling de Conexiones HTTP/2, Circuit Breaker, Paralelismo de Procesos CPU e Hilos I/O, Parámetros de Sitemaps y

Robots, Tarifas Públicas SIIU y Privadas, y APIs Externas). Todas las constantes, umbrales, timeouts y retardos admiten sobreescritura dinámica mediante variables de entorno del sistema operativo (`os.getenv`) analizadas mediante convertidores protegidos (`_safe_int`, `_safe_float`) para prevenir fallos en tiempo de arranque ante valores malformados.

- **downloader.py**: Implementa el cliente HTTP resiliente `RUCTDownloader` con el patrón *Circuit Breaker* y motor dual HTTP/2. Si acumula 10 fallos de red seguidos, se pausa automáticamente durante 5 minutos; si alcanza 3 pausas (15 minutos), lanza `SkipUniversityException` para omitir la universidad en conflicto sin colapsar el sistema. Incorpora:
 1. **Conexiones Multiplexadas HTTP/2 y Adaptador Transparente (`HTTP2ResponseWrapper`)**: Integración nativa de `httpx` con soporte HTTP/2 (`ENABLE_HTTP2=True`), multiplexación de flujos de datos, compresión binaria de cabeceras HPACK y negociación ALPN automática, con conmutación a HTTP/1.1 cuando el servidor no ofrece HTTP/2.
 2. **Reutilización de Sockets y Keep-Alive Pool**: Pool de conexiones configurado con `HTTP2_MAX_CONNECTIONS=20` y `HTTP2_MAX_KEEPALIVE_CONNECTIONS=10`, manteniendo canales TLS abiertos hacia los servidores del BOE y ministeriales sin forzar renegociaciones criptográficas continuas, con calentamiento de conexión optimizado sin peticiones redundantes.
 3. **Cerrojo de Concurrencia por Dominio (`DomainRateLimiter`)**: Sincronización mediante cerrojos de hilo por nombre de host (`_GLOBAL_DOMAIN_LOCKS`) que garantiza que ningún hilo emita peticiones concurrentes contra el mismo dominio universitario a la vez, garantizando un retardo cortés estricto (`REQUEST_DELAY = 0.35s + Jitter`).
 4. **Gestión Robusta de Reintentos y Retroceso Exponencial**: Manejo blindado de respuestas HTTP 429 (*Too Many Requests*) con pausas de retroceso exponencial aplicadas sobre el bucle externo de intentos, y reinicio limpio de la variable de respuesta por iteración para evitar registrar datos desactualizados en el ledger de auditoría.
 5. **Normalización de Enlaces y Verificación de Cabeceras**: Verifica la cabecera mágica `%PDF-` en streaming para descartar respuestas HTML erróneas de servidores oficiales, desinfecta esquemas duplicados (`http://https://`), eleva automáticamente a HTTPS los enlaces de boletines autonómicos y corrige erratas tipográficas del ministerio mediante `DOMAIN_MAPPINGS`.
- **parsers.py** y **boe_pdf_parser.py**: Analizan las fichas HTML del RUCT y procesan las resoluciones BOE con un modelo profundo del PDF. El modelo recorre cada página una vez y conserva texto, posiciones de palabras, tablas y delimitadores de titulación. Incorpora:
 1. **Caché LRU de Modelos de Documento en Memoria por SHA-256 (`_DOCUMENT_MODEL_CACHE`)**: Búfer thread-safe basado en `collections.OrderedDict` (con límite de 128 modelos) que almacena la estructura parseada de páginas y tablas indexada por la firma criptográfica SHA-256 del PDF. Cuando va-

rias titulaciones de una universidad comparten la misma orden ministerial del BOE, la extracción geométrica se ejecuta una única vez, reduciendo el tiempo de análisis de ≈ 347 ms a **0,92** ms en las consultas subsiguientes (aceleración de **376,3** $\times$).

2. **Prevención de Envenenamiento de Caché Negativa en Resoluciones Multi-Plan:** En `fase1_parte1_ruct_boe.py`, el registro de PDFs sin plan curricular en el checkpoint sólo se efectúa si el documento carece de tablas curriculares para cualquier grado (`document_has_any_curriculum = False`), evitando la inanición cruzada entre titulaciones que comparten la misma orden del BOE.
3. **Reconocimiento Multilingüe Ampliado y Acrónimos Cooficiales en Esquemas BOE:** Detección robusta de esquemas de tablas en castellano, catalán, valenciano, gallego, euskera e inglés (*Unitat Curricular, Malla Curricular, Baterako Irakasgaiak, Crédits ECTS, Tipologia, Temporalidad, Cuadrimestre*) con soporte en el parser dinámico textual para acrónimos normativos autonómicos e internacionales (MAL, AAL, KAN, TR, BA, OT, COMP, ELEC, INT, BST, MST).
4. **Inspección Priorizada de Candidatos BOE y Máscara Multi-Plan Robusta:** Ordenación por prioridad y fecha sin exclusión de resoluciones base cuando la modificación más reciente carece de tablas curriculares, junto a un algoritmo de delimitación vertical en PDFs multi-titulación inmune a la sobreescritura de estados entre grados colindantes.
5. **Validación Curricular Calibrada para Dobles Titulaciones y Dobles Másteres (`curriculum_validator.py`):** Evaluación jerárquica con discriminación previa de Másteres para evitar sobreexigencias erróneas de 300 ECTS en Dobles Másteres oficiales, validando correctamente planes de 90 a 150 ECTS.
6. **Algoritmo de Reconstrucción Multilínea de Asignaturas con Validación Combinada:** Unificación de nombres de asignaturas partidos en varias filas de tabla sin importar si las líneas secundarias comienzan en mayúsculas o números romanos (ej. *Ingeniería del Software + Avanzada, Estructuras de Datos y + Algoritmos II*), validando la integridad del nombre resultante.
7. **Extracción Geométrica Condicional en `pdfplumber`:** En lugar de calcular el posicionamiento espacial carácter a carácter en todo el documento, el motor evalúa rápidamente si la página contiene tablas o indicios curriculares (`RE_CURRICULAR_PAGE_HINTS`). En páginas administrativas de preámbulo o firmas, las líneas se derivan directamente desde `text.splitlines()`, ahorrando entre un 60 % y un 80 % de cómputo en Python puro.
8. **Gestión Segura de Recursos y Descriptores C++:** Envoltorio estricto `try...finally` alrededor de las instancias `pypdfium2.PdfDocument` en `ocr_parser.py` para invocar explícitamente `.close()` sobre los punteros nativos de C++, previniendo fugas de memoria y agotamiento de descriptores de archivo en el sistema operativo.

9. **Limpieza de Cerrojos Huérfanos (`error_logger.py`):** Detección proactiva de archivos de bloqueo `.lock` abandonados por interrupciones anómalas de proceso mediante comprobación de su marca temporal de modificación (`mtime > 60 s`), eliminándolos de forma segura para evitar estados permanentes de bloqueo (*deadlocks*).
 10. **Modelo profundo de documento y lectura OCR:** Reconstruye tablas delimitadas o sin bordes. Si el documento carece de capa de texto vectorial (`< 50` caracteres), delega condicionalmente en el motor OCR de `ocr_parser.py`.
 11. **Fusión Multi-BOE con Cuota por Curso y Saneamiento Universal:** Acumulación histórica con tope de 60 ECTS por curso, descarte de planes pre-Bolonia (anteriores a 2009), modelado específico de Doctorados (RD 99/2011) y limpieza sistemática (`unreverse_text` y saneamiento de espacios).
- **`fase1_parte1_ruct_boe.py`:** Orquesta la ejecución mediante una arquitectura paralela Productor-Consumidor con buffer híbrido en memoria (≤ 5 MB en RAM y volcado temporal con borrado garantizado para archivos mayores), reintentos ante contrapresión en colas (`queue.Full`) y reporte fiable de métricas de trabajadores en bloques `finally` ante paradas cooperativas.

Fase 1 - Parte 2: Recuperación desde Webs Oficiales

- **`fase1_parte2_web_crawler.py`:** Rastreado de portales oficiales para titulaciones sin plan BOE o con información incompleta.
- **Validación Curricular Calibrada y Completitud Normativa (`curriculum_validator.py`)**
Evaluación jerárquica conforme a los RD 822/2021 y RD 1393/2007, incorporando el estado `completo_normativo` para validar planes de estudio que listan todas las asignaturas troncales, obligatorias y TFG ($\geq 80\%$ de la carga total) respaldados por un cuadro resumen oficial verificado (≥ 240 ECTS en Grados o $\geq 60/90/120$ ECTS en Másteres), sin penalizar bolsas de optatividad no desglosadas.
- **Propagación Atómica Interuniversitaria y BOE Compartido:** Sincronización en cascada para más de 800 titulaciones conjuntas verificadas por ANECA y resoluciones ministeriales compartidas, aplicando salvaguardas de aislamiento institucional para impedir cualquier cruce entre titulaciones independientes de universidades distintas.
- **Extractor Universal de Tarjetas y Acordeones Web (`extract_subjects_from_card_blocks`)**
Soporte completo para maquetaciones universitarias modernas basadas en tarjetas visuales, listas y acordeones sin exigir prefijo numérico de código, identificando el nombre de la asignatura y deduciendo créditos, tipología y temporalidad a partir de etiquetas y contenedores adyacentes.
- **Validación Estructural de Tablas Curriculares y Extracción en Pestañas DOM:** Reconocimiento de tablas sin fila `<th>` explícita estructuradas ba-

jo encabezados de curso/cuatrimestre (`h1-h6`, `caption`, `legend`) y paneles DOM interactivos (`tab-pane`, `tabcontent`, `accordion`), con doble salvaguarda anti-ruido (descarte automático de tasas, calendarios y planes en extinción).

- **Normalización Bidireccional de Cuatrimestres y Marcadores Académicos Cooficiales:** Normalización léxica en `normalize_cuatrimestre` inmune al solapamiento con números ordinales de curso y ampliación de marcadores de URL en catalán, valenciano, gallego, euskera e inglés (`grau`, `graos`, `gradua`, `estudis`, `estudos`, `ikasketa`, `bachelor`, `undergraduate`).
- **Aislamiento de Banners Institucionales en la Validación de Grado:** Evaluación de tipología de ingeniería restringida al título principal (`<h1>`, `og:title`, `<title>`) para evitar que grados de ciencias o humanidades sean descartados erróneamente por banners de centros de ingeniería.
- **Índice Hub-and-Spoke con BFS multinivel y Cola $O(1)$:** Preindexación en cascada mediante `collections.deque` para garantizar extracción en tiempo constante ($O(1)$ en lugar de $O(N)$ de listas estándar) sobre catálogos maestros, facultades y portales de calidad, publicando evidencias estructuradas directamente en el ledger transaccional.
- **Extracción Robusta de Objetos JSON en Scripts (`spa_crawler.py`):** Algoritmo determinista de conteo de llaves y corchetes balanceados (`_extract_json_object`) que respeta cadenas entrecomilladas, sustituyendo expresiones regulares voraces (*greedy*) propensas a capturar JSONs inválidos. Manejo seguro de errores de navegación con re-propagación para evitar bloqueos por timeout en descargas de Playwright.
- **Gestión de Políticas Robots con Seguimiento de Redirecciones HTTPS:** En `robots_policy.py`, soporte para seguimiento automático de redirecciones 301/302 en el canal alternativo HTTPS y acotamiento de memoria en caché con desalojo LRU (`_MAX_CACHE_SIZE = 512`).
- **Protección contra Agotamiento de Memoria en PDFs Extensos:** Acotamiento estricto del número de páginas analizadas en el rastreador web (`_MAX_PDF_PAGES_EXTRACT = 80`) para impedir saturación de RAM ante catálogos o memorias universitarias voluminosas.
- **Disparo Condicional de Renderizado SPA y Extracción Privada:** Instanciación selectiva de Chromium sin cabeza únicamente cuando el HTML estático contiene < 3 asignaturas y extracción tarifaria oficial de universidades privadas sin inyección de datos ficticios.
- **Extractor Universal de Widgets Dinámicos y Endpoints REST (`extract_dynamic_widgets`):** Detección agnóstica de contenedores asíncronos en portales universitarios modernos que cargan sus mallas mediante llamadas AJAX/Fetch. El módulo inspecciona atributos HTML5 (`data-config`, `data-url`, `data-json`, etc.), extrae rutas relativas o canónicas hacia servicios REST institucionales, aplica validación estricta anti-SSRF para impedir peticiones a dominios externos no autorizados y parsea la respuesta JSON para extraer asignaturas, tipologías, cursos y enlaces a guías docentes.

- **Extracción Universal y Agnóstica de Programas de Doctorado Oficiales (RD 99/2011) (`extract_generic_doctoral_program`):** Marco de ingesta para los más de 3.500 programas oficiales de doctorado del sistema universitario español. Dado que bajo el RD 99/2011 los doctorados carecen de mallas lectivas con créditos ECTS tradicionales, el módulo descubre y navega de forma canónica hacia subpáginas de investigación (`lineas-de-investigacion`, `equipos-y-lineas`), descartando descargas directas de PDFs o información puramente administrativa. Extrae de forma multilingüe las líneas acreditadas a través de tres patrones universales (listas HTML adyacentes a encabezados semánticos, sub-encabezados H4/H5 en secciones y tablas estructuradas tipográficamente), captura la Escuela de Doctorado responsable y recopila las actividades formativas transversales, integrándolas en el campo estructurado `programa_doctoral` y como elementos curriculares con `caracter = "INVESTIGACION"`.

Fase 1 - Parte 3: Consolidación de Precios ECTS y Tarifas Públicas y Privadas (SIIU)

- `fase1_parte3_precios.py`: Asigna precios por crédito ECTS e importes anuales estimados a partir del catálogo local de referencias del SIIU, normativa autonómica disponible y prospectos de instituciones privadas.
- **Clasificación por Grados de Experimentalidad y Disciplinas (`classify_degree_experime`**
Diferenciación de precios por área de conocimiento (*Salud, Ciencias e Ingeniería, y Ciencias Sociales y Humanidades*):
 1. **Universidades Públicas:** Aplica las matrices de experimentalidad vigentes en los 17 decretos autonómicos y la UNED (`OFFICIAL_SIIU_PRICES_CATALOG`), calculando los precios de primera a cuarta matrícula.
 2. **Universidades Privadas:** Incorpora el catálogo estructurado `OFFICIAL_PRIVATE_UNIVERSI` para las 51 instituciones privadas del sistema universitario español (*UNAV, Comillas-ICADE/ICAI, UNIR, UOC, UAX, Nebrija, CEU San Pablo/Cardenal Herrera, Deusto, UEM, Loyola Andalucía, VIU, Isabel I, UDIMA, IE, CU-NEF, ESIC, etc.*), diferenciando los honorarios según la rama formativa (*Salud vs Ingenierías vs Sociales/Humanidades*) y el nivel de estudio.
- Implementa discriminación por categoría (*Grado, Máster Habilitante, Máster No Habilitante y Doctorado*) con la fórmula anual base $\text{Coste} = (60 \text{ ECTS} \times \text{Precio ECTS}) + \text{Tasas Admin.}$
- **Trazabilidad y Transparencia (`fuentes_precios`):** Registra explícitamente el origen normativo del importe (ej. *Decreto de Precios Públicos de la Junta de Andalucía (BOJA)* o *Tarifario Oficial Institución Privada - Universidad de Navarra*).
- Incorpora **caché en memoria** del catálogo `precios_ccaa.json` y volcado atómico mediante `atomic_json_dump` sobre los planes de estudio y el índice de titulaciones.

Fase 1 - Parte 4: Guías Docentes y Temarios EEES (`fase1_parte4_asignaturas.py`)

- `fase1_parte4_asignaturas.py`: Módulo especializado en la recolección profunda de guías docentes y temarios oficiales por asignatura en el Espacio Europeo de Educación Superior (EEES).
- **Descubrimiento Inteligente de URLs de Guía (`subject_guide_discovery.py`):** Sistema de scoring léxico y semántico con soporte adaptativo para asignaturas de término único o nombres compuestos, bonificación por coincidencias de ruta y verificación de tokens en co-oficiales e internacionales (`pla-docent`, `guia-docent`, `guies-docents`, `irakasgaiak`, `gida`, `teaching-guide`, `course-syllabus`).
- **Resolución de Identidad para Asignaturas de Término Único con Salvaguarda de Longitud:** Soporte para asociar guías a asignaturas de una sola palabra (*Física, Química, Álgebra, Cálculo, Biología, etc.*) con acotamiento estricto a títulos compuestos cortos (≤ 3 tokens), evitando rechazos en falso y protegiendo contra cruces de guías no afines.
- **Caché Jerárquica Dual con Caché Negativa Instantánea (`SubjectGuideCache`):**
 1. **Nivel L1 (Memoria RAM):** Diccionarios concurrentes protegidos por cerrojos reentrantes para resolución ultra-rápida durante la sesión activa.
 2. **Nivel L2 (Persistencia SQLite WAL):** Almacenamiento transaccional en `cache_guias_docentes.db` con tablas dedicadas para guías consolidadas (`guias_docentes`) y URLs descartadas con código 404/410 (`guias_negativas`).
 3. **Resolución Negativa a 0 ms:** Antes de despachar peticiones HTTP a la red, el resolutor de candidatas consulta la caché negativa. Enlaces no disponibles de facultades se omiten inmediatamente en 0 ms, ahorrando cientos de conexiones salientes por universidad.
- **Pipelines Multi-Estrategia de Análisis:**
 1. `parse_tabular_subject_guide`: Extrae guías con estructura tabular clásica (metadatos, departamentos, áreas de conocimiento, tablas `#temario` y `#procedimientos_evaluacion`).
 2. `parse_generic_eees_subject_guide`: Interpreta bloques temáticos semánticos en HTML (Requisitos Previos, Contenidos, Evaluación, Competencias, Bibliografía, Emails de Profesorado) con soporte multilingüe y extracción de porcentajes evaluativos por procesamiento de lenguaje natural en texto continuo.
 3. `parse_subject_guide_pdf_stream`: Procesa flujos de PDF en memoria RAM mediante `pypdf` y OCR condicional con `pypdfium2`.
- **Capa de Enriquecimiento y Normalización de Datos (`_enrich_parsed_guide`):**
 1. **Extracción de 8 Bloques Canónicos Semánticos en Prosa Completa:** Preservación íntegra de la narrativa docente en los campos clave (`ficha_identificativa`, `profesorado_texto`, `prerrequisitos_texto`, `competencias_texto`, `temario_texto`, `metodologia_texto`, `criterios_evaluacion` y `bibliografia_texto`),

garantizando que temarios extensos no se trunquen ni dependan exclusivamente de listados estructurados.

2. **Desglose Estructurado de Evaluación Multilingüe (`_normalize_evaluation_breakdown`):**

Extrae y normaliza los porcentajes ponderados de evaluación en cuatro métricas fijas (`teoria_porcentaje`, `practicas_porcentaje`, `evaluacion_continua_porcentaje` y `examen_final_porcentaje`) reconociendo vocabulario evaluativo en castellano, catalán/valenciano, gallego, euskera e inglés.

3. **Detección Automática de Lengua Docente (`_infer_subject_guide_language`):**

Analiza el léxico y marcadores ortográficos del temario para clasificar la asignatura en *Castellano*, *Català*, *Galego*, *Euskara* o *English*.

■ **Protección Universal de Subdominios y Resolución DNS (`downloader.py`):**

Aislamiento de hosts de segundo nivel frente a subdominios institucionales de 3 o más niveles (`asignaturas.uca.es`, `guiasdocentes.ucm.es`, `sia.unican.es`, `guies.uab.cat`). La función `_request_url_variants` inhibe la adición indebida de prefijos `www.` sobre subdominios activos, eliminando falsos fallos DNS en la verificación de `robots.txt` y garantizando la conexión limpia con los portales de gestión docente de todo el sistema universitario.

■ **Enriquecedor Universal de Códigos Web (`_enrich_degree_subject_codes_and_urls`):**

Puente de enlace entre resoluciones oficiales del BOE y los portales web de facultades y escuelas universitarias. Cuando un plan procedente del BOE carece de códigos numéricos de asignatura, el sistema rastrea las tablas curriculares de los centros y asocia los códigos internos (4 a 8 dígitos) y las URLs canónicas de las guías mediante cruce semántico de títulos con normalización Unicode NFKD y solapamiento léxico de tokens.

■ **Tolerancia a Periodos de Transición Académica (`year_candidates`):** Reconocimiento adaptativo de guías vigentes del curso de respaldo inmediato (2025-26) durante periodos estivales y de matriculación temprana, asegurando la disponibilidad ininterrumpida de temarios y metodologías docentes.

■ **Orquestación Modular y Suite de Filtros CLI (`main_fase_1.py`):** Proporciona una interfaz unificada por consola con soporte para ejecución segmentada por fases (`--parte`, `--parts`, `--all`), filtrado por universidades (`--univs` por código o nombre), segmentación por titularidad (`--publicas`, `--privadas`, `--tipo-univ`) y filtrado semántico de planes de estudio por subcadena en el título de la titulación (`--grado` "Informática").

■ **Aislamiento y Ejecución Segura en Docker con Playwright Headless:** Integración de perfiles de usuario sin privilegios (`crawler:crawler`) con directorio `$HOME=/home/crawler` y binario Chromium optimizado para ejecución desatendida sin incidencias de permisos ni bloqueos por fallos de socket.

■ **Emisión de Telemetría con Concurrencia Acotada (`progress_emitter.py`):**

Desacoplamiento del envío de métricas en vivo hacia la API mediante una cola fija `queue.Queue(maxsize=16)` procesada por un hilo demonio exclusivo, eliminando la creación descontrolada de hilos durante los volcados de progreso.

Gestor de Estado Incremental (**CheckpointManager**)

- **checkpoint.py**: Gestor atómico del estado del rastreo. Mantiene el estado de reanudación en `checkpoint.json` y utiliza SQLite WAL para el ledger de peticiones y la caché. La escritura se agrupa en puntos de control para evitar operaciones de disco innecesarias.

Mecanismos de Rendimiento, Reanudación y Cortesía

La Fase 1 incorpora mecanismos orientados a reducir trabajo repetido sin alterar la semántica de los datos ni el comportamiento respetuoso frente a los servidores externos:

- **Paralelismo acotado y Productor-Consumidor**: La Parte 1 desacopla descargas y análisis de PDF mediante trabajadores configurables. La precarga RUCT se limita por `ASYNC_PREFETCH_WORKERS` y `RUCT_PREFETCH_LOOKAHEAD`.
- **Transferencia híbrida y Caché SHA-256 de PDF**: Los documentos pequeños se procesan en memoria y la caché LRU evita reanalizar el mismo PDF cuando contiene varios planes.
- **Modelo profundo y Extracción Geométrica Condicional**: El parser analiza la geometría de palabras solo en páginas relevantes del PDF, reservando el análisis plano para preámbulos.
- **Caché y ledger**: Las respuestas HTTP, sus validadores y huellas SHA-256 se registran en SQLite WAL. Esto permite reanudar y revalidar fuentes sin repetir solicitudes cuando la caché es válida.
- **Caché negativa de Guías Docentes**: Omite peticiones a enlaces 404/410 en 0 ms desde el almacén SQLite WAL.
- **Rastreo web responsable y Circuit Breaker**: Las Partes 2 y 4 agrupan el trabajo por universidad, consultan `robots.txt` con caché LRU, respetan retardos por dominio y aíslan temporalmente dominios que registran incidencias continuadas.
- **Renderizado condicional**: Playwright se reserva para portales cuyo HTML estático no aporta información curricular suficiente (< 3 asignaturas).

7.2.2. Fase 2: Módulo de API REST y Persistencia (**Codigo/API/**)

- **database/schema.sql**: Define el esquema relacional en PostgreSQL. Activa la extensión de trigramas `pg_trgm` con índices GIN en columnas de búsqueda (`nombre`, `titulo`) y crea el rol restringido `unihub_api_user`. Incorpora las columnas de trazabilidad y calidad `origen_fuente`, `pdf_sha256`, `estado_calidad`, `motivos_calidad`, `fuentes_verificadas_urls`, `tipos_estructura` y la columna estructurada `programa_doctoral_JSONB` en la tabla `planes_estudio`.

- `connection.py`: Gestiona el conjunto de conexiones SQLAlchemy (`pool_size=15`, `max_overflow=25`, `pool_pre_ping=True`).
- `security.py`: Control de acceso administrativo para la cabecera HTTP X-API-Key. Valida la clave mediante `secrets.compare_digest(api_key, settings.ADMIN_API_KEY)`, inmune a ataques de tiempo (timing attacks) por canal lateral.
- `etl_loader.py`: Proceso de ingesta de datos JSON hacia la base de datos utilizando `bulk_save_objects` para acelerar las inserciones masivas de asignaturas (reduciendo el tiempo de 15s a menos de 1.5s). Soporta rutas adaptativas tanto en entorno nativo como contenerizado (`/app/Datos`). Incorpora un mecanismo de sincronización reactiva en segundo plano con control de concurrencia mediante archivo de cerrojo de proceso PID (`etl_running.lock` en `tempfile.gettempdir()`) y comprobación de PID activo con `os.kill(pid, 0)`. Incluye un umbral de seguridad de representatividad (≥ 100 titulaciones) para prevenir borrados en cascada accidentales ante volcados parciales del crawler. Reconoce la tipología `programa_doctorado_investigacion` y estados autorizados (`doctorado_verificado`, `doctorado_estructural`, `doctorado_oficial`, `verificado_programa_doctoral`), volcando el objeto `programa_doctoral` e insertando las líneas de investigación como elementos curriculares con `caracter = 'INVESTIGACION'`.
- `routes/`: Define los controladores REST para universidades (`universidades.py`), titulaciones y planes (`titulaciones.py`) y telemetría (`estadisticas.py`). Destaca el soporte para filtrado vocacional por Rama de Conocimiento (`rama: sociales, ingenieria, salud, humanidades, ciencias`), exposición estructurada de programas de doctorado en `PlanEstudiosOut`, el endpoint GET `/api/v1/estadisticas/cobertura` que expone la cobertura global y el validador GET `/api/v1/auth/verify`.

7.2.3. Fase 3: Módulo de Interfaz Web y Calculadora (Codigo/WWW/)

- `src/components/ErrorBoundary.jsx`: Componente de captura de errores de React que muestra una interfaz de respaldo y evita el colapso total de la aplicación ante fallos en subárboles.
- `src/components/AdminLogin.jsx`: Acceso asegurado con validación asíncrona en tiempo real contra el endpoint `/auth/verify` del backend antes de transicionar al panel de control, con indicador de carga animado.
- Carga diferida de rutas pesadas con `React.lazy` y `Suspense`: `AdminDashboard`, `AdminLogin`, `Geolocation`, `TuitionCalculator`, `AboutUs`, `Pagination` y `Footer` se cargan bajo demanda para reducir el bundle inicial.
- `src/services/api.js`: Cliente de comunicación HTTP con la API REST con interceptor de latencias `perfTracker`, propagación del filtro `rama`, filtrado de excepciones `AbortError` (para no distorsionar métricas en búsquedas con `*debounce*`) y formateo corregido de CCAA.
- `src/App.jsx`: Enrutamiento SPA sincronizado con la **History API** del navegador (`pushState` y escucha de eventos `popstate`), permitiendo la navegación fluida y el soporte nativo del botón atrás/adelante. Incorpora el banner interactivo de

desconexión (*Offline Alert Banner*) con detección en tiempo real de fallos de red hacia el backend y botón de reintento de conexión inmediato.

- `src/components/PlanModal.jsx`: Modal curricular enriquecido con soporte dual: (1) desglose de asignaturas ECTS organizadas por curso/cuatrimestre con resalta-do visual de **Menciones Curriculares e Itinerarios Oficiales** ([Mención...]) para más de 420 grados; y (2) panel interactivo para **Programas Oficiales de Doctorado e Investigación (RD 99/2011)**, que renderiza la insignia de cali-dad oficial verificada, la Escuela de Doctorado tutelar, una cuadrícula interactiva de **Líneas de Investigación Científica Oficiales**, listado de Actividades For-mativas Transversales, régimen económico de Tutela Académica Anual y enlace oficial directo. Bloqueo de scroll en el body (`overflow: hidden`) para ergonomía modal.
- `src/components/Navbar.jsx`: Barra de navegación accesible con soporte para menú hamburguesa táctil colapsable en dispositivos móviles y conmutador de modo claro/oscuro.
- `src/components/TuitionCalculator.jsx`: Motor de simulación financiera de matrícu-la universitaria. Implementa multiplicadores por repetición de asignatura (1,0×, 1,5×, 3,0×, 4,5×) selección rápida por curso, preselección segura hasta 60 ECTS para planes planos, soporte para universidades públicas y privadas, y el sistema completo de exencio-nes sociales (Beca MEC, Familia Numerosa, Discapacidad $\geq 33\%$, Bonificación 99% CCAA y Matrícula de Honor).
- `src/components/Geolocation.jsx`: Búsqueda de universidades por distancia Ha-versine a más de 50 ciudades y capitales de provincia de España o ubicación GPS con reinicio automático de paginación.
- `src/components/AboutUs.jsx`: Sección informativa ampliada con el desglose técni-co de las 4 Fases, datos institucionales del TFG de la UCA y enlaces interactivos a fuentes oficiales.
- `src/components/AdminDashboard.jsx`: Panel de control avanzado asegurado con exportación masiva CRUD protegida (Blob + URL.createObjectURL y sanitiza-ción anti CSV Injection), semáforos Core Web Vitals, barras animadas de Docker, telemetría Green IT y monitor de sincronización ETL en vivo.
- `src/components/DegreeCard.jsx` y `UnivCard.jsx`: Componentes visuales me-moizados con soporte de accesibilidad por teclado (`tabIndex={0}`), eventos de tecla Enter/Espacio), parseo numérico protegido y cálculo instantáneo de costes de 1^a a 4^a matrícula.
- `src/index.css`: Sistema de diseño basado en la identidad corporativa de la Uni-versidad de Cádiz (UCA) con variables CSS, efectos *glassmorphism*, scrollbar estándar multiplataforma (`scrollbar-width: thin`) y adaptabilidad responsive integral.

7.2.4. Fase 4: Despliegue Contenerizado y Orquestación (Codigo/Docker/)

- `docker-compose.yml`: Define la red interna `unihub_network`, el volumen persistente `unihub_postgres_data` y los 4 servicios (`db`, `crawler`, `api`, `www`). Incluye `healthcheck` con `pg_isready` para la base de datos y verificación de estado en la API (`/api/v1/salud`); el servicio web depende del estado saludable declarado por la API (`condition: service_healthy`).
- `api/Dockerfile`: Imagen base `python:3.12-slim`. Instala dependencias de sistema `postgresql-client`, `libpq-dev`, `gcc`, `g++` y copia `Codigo/API` y `Codigo/Crawler/Datos` al contenedor.
- `crawler/Dockerfile`: Imagen base Python con volumen montado `/app/Datos`. Copia todo el crawler para ejecutar `main.py` de forma programada o manual.
- `www/Dockerfile`: Construcción multi-etapa Node 20-alpine para el build Vite y Nginx 1.25-alpine para servir los artefactos estáticos. Variables `VITE_API_URL`, `VITE_ADMIN_USER`, `VITE_ADMIN_FEES` inyectadas en compilación.
- `entrypoint.sh`: Script de arranque que comprueba la disponibilidad de PostgreSQL mediante `nc` y lanza el servidor FastAPI con `uvicorn main:app --host 0.0.0.0 --port 8000`.

7.2.5. Pruebas y Auditoría de Rendimiento (Codigo/Pruebas/)

- `run_phase1_detailed_test.py`: Prueba de trazabilidad y auditoría sobre 5 universidades de referencia (3 públicas y 2 privadas). Ejecuta Parte 1 RUCT+BOE PDF, Parte 2 rescate web oficial y Wikidata, Parte 3 cálculo de precios ECTS. Genera informe Markdown y JSON con métricas de cobertura, asignaturas extraídas y auditoría de redundancias.
- Detección de redundancias: RED-01 normalización duplicada de URLs en `downloader.py` y `parsers.py`; RED-02 lecturas repetidas de `plan_file` en `univ_web_crawler.py`, `main.py` y `precios_crawler.py`; RED-03 recálculo de precios en múltiples pasos.
- Documentación de pruebas: `EstudioRendimientoFase1.md`, `EstudioTasaAcuerdoReal.md`, `ResultadosPruebaFase1.md/json`.

7.3. Esquema Físico Relacional y Modelo de Persistencia

El esquema físico en PostgreSQL se diseña para garantizar la integridad referencial, la indexación optimizada de consultas y el aislamiento estricto de privilegios de acceso. El modelo relacional implementado se articula en torno a tres entidades principales y sus correspondientes tablas de soporte:

1. **Entidad universidades:** Almacena la identidad institucional del centro de educación superior (`codigo` oficial RUCT como clave primaria, denominación legal,

tipología pública o privada, comunidad autónoma, provincia, municipio, portal web institucional y canales de contacto).

2. **Entidad titulaciones:** Registra la oferta académica oficial vinculada a cada universidad (`codigo_estudio` único, nivel académico Grado, Máster o Doctorado, estado de vigencia, precios regulados SIIU por crédito ECTS de primera a cuarta matrícula y coste anual estimado).
3. **Entidad planes estudio:** Almacena la trazabilidad de la publicación oficial en el BOE (enlace web a la resolución, fecha de publicación, huella criptográfica SHA-256 del PDF original y estado de calidad curricular). Asimismo, para las titulaciones de Doctorado conforme al RD 99/2011, incorpora la columna semiestructurada `programa_doctoral` JSONB, que modela formalmente las líneas de investigación científica acreditadas, la Escuela de Doctorado responsable y el inventario de actividades formativas transversales.
4. **Estructura Curricular Desglosada (`resumen_creditos` y `elementos_curriculares`):** Tablas dependientes con borrado en cascada (`ON DELETE CASCADE`) que descomponen el plan de estudios en módulos, materias y asignaturas, registrando créditos ECTS, carácter (Básica, Obligatoria, Optativa, Prácticas o Trabajo Fin de Título), temporalidad (curso y cuatrimestre) y guías docentes asociadas.
5. **Auditoría de Ingesta y Salud del Sistema (`errores_crawler` y `estadisticas_rendimiento`):** Registros relacionales para la auditoría forense de incidencias de red y métricas de rendimiento por fase (duración, consumo de memoria RSS y tasa de acierto).
6. **Aislamiento de Privilegios de Acceso (RBAC):** Creación dinámica en PostgreSQL del rol de solo lectura `unihub_api_user`, al que se asigna exclusivamente el privilegio `GRANT SELECT` sobre las tablas del esquema público, blindando la base de datos frente a mutaciones accidentales o vulnerabilidades de inyección procedentes del servicio REST estándar.

Siguiendo las directrices metodológicas de documentación técnica, el código fuente DDL completo que materializa este diseño en PostgreSQL (`schema.sql`) se ha trasladado al Anexo D (Esquema Físico DDL de Base de Datos), donde se detallan todas las sentencias de creación de tablas, índices B-Tree y GIN con trigramas (`pg_trgm`) y directivas de seguridad.

8. Pruebas del Sistema

Este capítulo describe la estrategia de pruebas aplicada a **UniHub**. Se distingue entre pruebas deterministas, que se ejecutan sin depender de fuentes externas, y evaluaciones empíricas, que contrastan el resultado con publicaciones oficiales. Esta separación evita confundir una prueba de software con una medición de cobertura nacional.

8.1. Estrategia de Pruebas

La validación se organiza en cuatro niveles: pruebas unitarias de normalización y reglas de negocio; pruebas de integración entre crawler, persistencia, API y cliente web; pruebas de regresión del parser BOE; y evaluaciones empíricas contra documentos oficiales. Las pruebas con red se ejecutan explícitamente, de forma secuencial y respetando los retardos y las restricciones de acceso de las fuentes consultadas.

8.2. Pruebas Unitarias y de Integración

La suite automatizada cubre de forma exhaustiva los contratos funcionales del sistema mediante más de 30 módulos de prueba especializados:

- **Ingesta RUCT, BOE y Persistencia Dual:** Clasificación de titulaciones, filtro de vigencia, persistencia transaccional SQLite WAL y recuperación ante corrupción (`test_checkpoint_resilience.py`, `test_crawl_ledger_resilience.py`, `test_cache_recovery.py`).
- **Políticas de Red, HTTP/2 y Cortesía:** Normalización canónica de URLs, verificación de cabeceras PDF, política `robots.txt` con seguimiento de redirecciones HTTPS, reintentos exponenciales ante HTTP 429 y aislamiento de dominio mediante *Circuit Breaker* (`test_downloader_domain_boundaries.py`, `test_crawler_politeness.py`, `test_robots_policy_redirects.py`, `test_http_cache_ttl.py`).
- **Guías Docentes y Temarios EEES:** Análisis de estructuras tabulares, extracción semántica multilingüe, normalización porcentual del desglose de evaluación, detección automática de lengua docente y resolución negativa de candidatas en 0 ms (`test_subject_guide_crawler.py`, `test_subject_guide_discovery_cache.py`, `test_phase4_resume.py`, `test_phase4_metrics_aggregation.py`).
- **Auditoría de Procesos y Concurrency:** Gestión segura de descriptores nativos C++, recolección de cerrojos huérfanos por marca temporal, cola acotada de telemetría y algoritmos de balanceo de llaves en scripts SPA (`test_phase1_audit_fixes.py`, `test_phase1_audit_v2_fixes.py`, `test_spa_render_optimizations.py`).

- **Cálculo Tarifario y Servicios REST:** Consolidación de precios autonómicos SIIU, carga masiva transaccional ETL, contratos de calidad de planes y endpoints de la API (`test_plan_publication_api.py`, `test_quality_persistence_contract.py`).
- **Widgets Dinámicos y Endpoints REST:** Detección agnóstica de atributos HTML5 (`data-config`, `data-url`), extracción de rutas REST y prevención SSRF contra dominios externos (`test_generic_dynamic_widgets.py`).
- **Programas Oficiales de Doctorado (RD 99/2011):** Verificación de los tres patrones universales (listas, subtítulos, tablas), descarte de ruido administrativo, detección multilingüe de escuela y validación curricular sin créditos ECTS lectivos (`test_doctoral_extraction.py`).
- **Frontend y Despliegue:** Representación de datos en la SPA, control de accesos RBAC y contratos de orquestación contenerizada (`test_frontend_data_representation.py`).

La ejecución automatizada de la suite completa se realiza mediante el comando:

```
python -m unittest discover -s Codigo/Pruebas -p "test_*.py"
```

El resultado de la verificación es de **407 pruebas superadas** y **una prueba omitida** de forma explícita (evaluación empírica con descarga de red en vivo), completándose la ejecución íntegra en apenas **37.3 segundos** con cero fallos y cero regresiones funcionales.

8.3. Validación del Parser de Resoluciones BOE

El parser se valida tanto con fixtures sin red como con publicaciones oficiales. La prueba `test_boe_deep_parser.py` cubre cuatro contratos críticos de alto riesgo:

1. tabla curricular convencional con cabecera explícita;
2. tabla sin bordes recuperada desde líneas y coordenadas espaciales del PDF;
3. resolución con dos titulaciones en la misma página, verificando que sólo se conserve la sección solicitada;
4. aislamiento de la caché en memoria de modelos de documento SHA-256 (`_DOCUMENT_MODEL_CACHE`) para garantizar independencia entre ejecuciones de prueba.

Además, las pruebas de regresión verifican encabezados multilingües, filas optativas, temporalidad, créditos declarados y documentos multi-titulación. Estas pruebas detectan cambios de comportamiento, pero no miden por sí solas la precisión frente a la fuente publicada.

8.3.1. Evaluación Empírica Reproducible

El script `boe_empirical_evaluation.py` contrasta el PDF consumido por el crawler con la tabla HTML estructurada del BOE. La muestra contiene cinco resolu-

ciones: BOE-A-2022-12382, BOE-A-2024-1769, BOE-A-2023-6963, BOE-A-2023-11558 y BOE-A-2025-21807.

En la ejecución de referencia se compararon 181 asignaturas: la precisión fue del 100 %, la cobertura del 100 % y los créditos ECTS declarados coincidieron en los cinco casos. La muestra incluye Grados, Másteres y una resolución multi-titulación. Este resultado evidencia que el parser funciona en los casos ensayados; no debe extrapolarse como una garantía estadística para todos los PDFs históricos o para documentos que no publiquen el detalle curricular.

8.4. Pruebas de Rendimiento y Benchmarking

Para evaluar el impacto de las optimizaciones arquitectónicas en el análisis de documentos PDF, se ejecutó un benchmark comparativo de rendimiento (`benchmark_pdf_optimizations`).

- **Pase en Frío (Extracción Geométrica Condicional):** El análisis de un PDF de varias páginas con filtrado previo de páginas irrelevantes toma $\approx 347,2$ ms.
- **Pase en Caliente (Caché SHA-256 de Modelos de Documento):** Cuando una segunda titulación comparte el mismo documento oficial del BOE, la recuperación instantánea de la estructura geométrica desde memoria RAM toma únicamente **0,92** ms.
- **Ganancia de Eficiencia:** Se obtiene un factor de aceleración de **376,3** $\times$ (reducción del 99.7 % del tiempo de CPU en documentos compartidos), permitiendo que la suite completa de 407 tests se ejecute con máxima rapidez y solidez.

8.5. Pruebas de Ejecución Controlada

Las ejecuciones limitadas del crawler se emplean para comprobar que el pipeline:

- reanuda desde `checkpoint.json` sin destruir información previa;
- registra respuestas, errores y caché en el ledger SQLite;
- respeta la política de acceso por dominio y no trata una denegación de `robots.txt` como error recuperable mediante fuerza bruta;
- descarta URLs candidatas inválidas (404/410) en 0 ms a través del índice negativo SQLite WAL;
- conserva los resultados incompletos con su estado de fuente, en lugar de rellenarlos con datos no publicados.

La duración y la cobertura de una ejecución nacional dependen de las respuestas de RUCT, BOE y las universidades. Por ello, cualquier resultado operativo debe acompañarse del manifiesto de ejecución, el intervalo temporal, los límites de universidades/titulaciones y la configuración de trabajadores utilizada.

8.6. Pruebas No Funcionales

- **Robustez:** cada titulación se procesa de forma aislada; una incidencia de red o de parseo queda registrada sin invalidar el resto de la ejecución.
- **Gestión de Recursos y Fugas:** verificación del cierre explícito de descriptores de sockets, conexiones de bases de datos y punteros nativos C++ en librerías de renderizado.
- **Cortesía:** el crawler consulta `robots.txt`, aplica retardos por dominio y limita la concurrencia sobre una misma institución.
- **Trazabilidad:** los planes conservan URL, procedencia, fecha de comprobación y, cuando corresponde, la huella SHA-256 del PDF.
- **Rendimiento:** las métricas de tiempo, memoria y caché se registran durante la ejecución. Deben presentarse como mediciones asociadas a un manifiesto, no como constantes universales del sistema.

8.7. Criterio de Aceptación

Una versión candidata se acepta cuando la suite determinista finaliza correctamente (100 % de tests superados), la evaluación empírica BOE no reduce precisión ni cobertura respecto a la referencia y las pruebas de integración no detectan roturas de contrato entre crawler, API y frontend. Las titulaciones sin detalle oficial permanecen identificadas como incompletas; la ausencia de datos no se transforma en un dato estimado.

9. Despliegue del Sistema

Este capítulo recoge la arquitectura física planteada para el sistema **UniHub**, las instrucciones para su despliegue y las instrucciones para la operación y mantenimiento del nivel de servicio.

9.1. Arquitectura Física

La arquitectura física del sistema se basa en la virtualización liviana mediante contenedores Docker sobre un servidor anfitrión con sistema operativo Linux o Windows con soporte para Docker Engine. La infraestructura se compone de 4 contenedores aislados que se comunican en una red puente virtualizada (**unihub_network**):

1. **Servidor de Base de Datos (unihub_db)**: Contenedor PostgreSQL 15 escuchando en el puerto 5432 con volumen nominado **unihub_postgres_data** y prueba de salud (*healthcheck*) configurada con **start_period: 20s** y 10 reintentos para tolerar la carga DDL inicial en instalaciones limpias.
2. **Servidor de API REST (unihub_api)**: Contenedor Python 3.12 / FastAPI escuchando en el puerto 8000 con soporte para sincronización ETL reactiva (**/api/v1/admin/sync-etl**).
3. **Servidor Web Nginx (unihub_www)**: Contenedor Nginx expuesto en el puerto configurado por **WEB_PORT**, que sirve la SPA compilada en React.
4. **Contenedor de Ingesta (unihub_crawler)**: Contenedor Python para ejecuciones del rastreador, con montaje directo de **Codigo/Crawler/Datos**. El calendario Cron se configura explícitamente mediante **CRAWLER_CRON_SCHEDULE**; el valor de referencia del **entrypoint** es **0 2 1 * ***.

9.2. Instrucciones de despliegue

9.2.1. Requisitos previos

- Docker Engine 20.10+ y Docker Compose v2.
- Al menos 2 GB de memoria RAM libre y 5 GB de espacio en disco.

9.2.2. Inventario de componentes

Los artefactos del despliegue se ubican en **Codigo/Docker/**:

- **docker-compose.yml**

- crawler/Dockerfile y crawler/entrypoint.sh
- api/Dockerfile y api/entrypoint.sh
- www/Dockerfile y www/nginx.conf
- .dockerignore (raíz del proyecto): Excluye contextos pesados de compilación (como la carpeta de PDFs) para agilizar drásticamente el levantamiento de los contenedores.
- Archivo .env.example con las variables de entorno requeridas por Docker Compose.

9.2.3. Procedimientos de instalación

Para desplegar el sistema contenerizado se dispone de dos modalidades:

Modalidad A: Mediante Script Automatizado (Recomendado)

Ejecutar desde la raíz del proyecto el script de conveniencia:

```
iniciar_proyecto.bat
```

Dicho script verifica los puertos del sistema, compila las imágenes Docker y levanta los 4 contenedores automáticamente.

Modalidad B: Despliegue General con Docker Compose

1. Navegar a la carpeta del entorno Docker desde la raíz del proyecto:

```
cd Codigo/Docker
```

2. Construir y levantar todos los servicios en segundo plano:

```
docker compose up --build -d
```

3. Alternativamente, para un despliegue selectivo (sin el crawler de la Fase 1):

```
docker compose up -d db api www
```

4. **Control de Ejecución del Crawler al Arranque:** En el archivo .env se puede configurar CRAWLER_RUN_ON_STARTUP=false para omitir el rastreo automático al iniciar los contenedores, o bien definir argumentos específicos en CRAWLER_PARTS_ARGS (por ejemplo, --parte 4 --universities 005).

Construcción de Imágenes y Variables de Entorno

La compilación del contenedor Frontend (`unihub_www`) utiliza un Dockerfile de dos fases (*Multi-stage build*) basado en `node:20-alpine` y `nginx:1.25-alpine`. Durante la fase de construcción de Node.js, se inyecta dinámicamente la variable de entorno `VITE_API_URL` mediante argumentos de compilación (ARG) definidos en el archivo `docker-compose.yml`. Esto permite a la aplicación React compilar de forma estática la ruta a la API y operar transparentemente a través del enrutador web.

9.2.4. Pruebas de implantación

Acceder desde el navegador web a las siguientes direcciones de verificación:

- Portal Web Frontend UniHub: <http://localhost> (producción Nginx) o <http://localhost:5173> (servidor de desarrollo Vite).
- Documentación API Swagger: <http://localhost/docs>
- Documentación ReDoc: <http://localhost/redoc>
- Panel de Administración (Acceso Aislado): <http://localhost/admin>

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio

Para garantizar la continuidad de servicio y la persistencia de los datos:

- **Chequeo de Logs:** Monitorear la salud de los servicios con `docker compose logs -f`.
- **Telemetría cgroup y Green IT:** La API expone en `/api/v1/estadisticas/contenedores` la monitorización de RAM RSS/Peak, uso de CPU %, espacio en disco y estimación de Huella de Carbono (gCO_2).
- **Copia de Seguridad de la BD:** Exportar backup mediante `docker compose exec db pg_dump -U postgres unihub_db > backup.sql`.
- **Copia de Seguridad de Archivos JSON:** Respalidar la carpeta `Codigo/Crawler/Datos` en el host. El número de archivos es variable y debe obtenerse del manifiesto de la ejecución que se quiera auditar.

Parte III

Epílogo

10. Conclusiones

En este último capítulo se detallan los objetivos alcanzados en el proyecto **UniHub**, las lecciones aprendidas tras el desarrollo y se identifican las oportunidades de mejora sobre el software desarrollado.

10.1. Objetivos alcanzados

Se han implementado los objetivos funcionales definidos para el proyecto a lo largo de sus cuatro fases de ingeniería:

1. Construcción de un rastreador modular en Python (Fase 1) organizado en cuatro partes: RUCT y BOE, recuperación desde webs oficiales, precios ECTS y guías docentes. El proceso clasifica Doctorados, aplica filtros de vigencia, registra incidencias y conserva trazabilidad de fuentes. El parser BOE delimita la titulación dentro del PDF y analiza sus tablas y líneas posicionadas sin generar materias que no estén publicadas.
2. Implementación de un modelo relacional en PostgreSQL (Fase 2) con control de acceso basado en roles (RBAC) aislando al usuario público (`unihub_api_user`), protección de operaciones críticas mediante `X-API-Key`, pipeline ETL, ordenación prioritaria (Públicas primero, Privadas después), soporte de métricas físicas de contenedores cgroup y servicio REST en FastAPI.
3. Creación de una aplicación web SPA en React (Fase 3) bajo el sistema de diseño e identidad visual de la Universidad de Cádiz (UCA). Incluye paginación configurable, geolocalización por fórmula de Haversine, un **Motor de Simulación Financiera (Calculadora de Matrícula)** con soporte para leyes de precios públicos autonómicos y becas, y un panel de administración asegurado para la monitorización ETL y Docker.
4. Orquestación completa en contenedores Docker independientes (Fase 4) optimizada mediante *multi-stage builds* e inyección dinámica de variables de compilación (`VITE_API_URL`), reduciendo drásticamente el peso de imagen (vía `.dockerignore`) y garantizando la persistencia mediante volúmenes nominados.

10.1.1. Resultados Cuantitativos y Métricas Clave

Las métricas del sistema se obtienen a partir del manifiesto, los puntos de control y la base de datos de cada ejecución. Por ello deben presentarse con fecha, configuración y alcance explícitos; no se consideran propiedades permanentes del software.

- **Cobertura del catálogo:** El catálogo nacional se procesa por universidad y titulación, y la cobertura debe calcularse sobre el conjunto efectivamente finalizado

en el manifiesto de ejecución.

- **Calidad curricular y normalización:** Un plan se clasifica según la fuente disponible, la presencia de elementos curriculares y la coherencia de sus créditos. La capa de enriquecimiento infiere la lengua académica de impartición y normaliza el desglose porcentual de los criterios de evaluación.
- **Validación empírica del parser:** La evaluación de cinco resoluciones BOE de referencia contrastó 181 asignaturas con sus tablas oficiales y obtuvo precisión y cobertura del 100 % en esa muestra. Este resultado valida los casos ensayados, pero no sustituye una auditoría censal de todos los formatos de publicación.
- **Rendimiento, caché y sostenibilidad:** Las optimizaciones algorítmicas implementadas (caché SHA-256 de modelos de documento y extracción condicional de geometría) aportan una aceleración de hasta **376,3×** en documentos compartidos. Junto con la caché negativa SQLite WAL para guías docentes a 0 ms, permiten ejecutar la suite íntegra de **392 pruebas automatizadas en 28.1 segundos**, reduciendo significativamente el consumo de CPU y la huella energética global.

10.2. Lecciones aprendidas

Entre las principales lecciones aprendidas destacan:

- **Tolerancia a Fallos y Resiliencia en Scraping Web:** La importancia de aislar excepciones por registro e implementar pausas estratégicas de resiliencia (5m tras 10 fallos y cortocircuito a los 15m) con registro estructurado en `errores_crawler.json`.
- **Gestión Estricta de la Integridad de Datos:** La necesidad de filtrar titulaciones extinguidas y descartar valores indeterminados (`, undefined"`) para prevenir la corrupción del repositorio histórico de datos.
- **Diseño Orientado al Usuario y Accesibilidad:** El uso de un sistema de diseño estructurado (colores corporativos UCA, badge rosa para Doctorados) y elementos modulados de paginación para ofrecer una experiencia intuitiva.
- **Arquitectura de Contenerización Aislada:** La configuración adecuada de redes puente virtualizadas y volúmenes nominados para asegurar el desacoplamiento de servicios y la persistencia de datos.

10.3. Trabajo futuro

Como líneas de desarrollo futuro se proponen:

- Ampliar la evaluación empírica del parser con una muestra estratificada de resoluciones BOE, documentos escaneados y publicaciones con varias titulaciones.

- Incorporar modelos de lenguaje (LLMs) para el análisis semántico avanzado de competencias en las guías docentes, con validación humana y trazabilidad de la fuente.
- Implementar notificaciones automáticas por correo electrónico cuando una titulación sufra una modificación oficial publicada en el BOE.
- Ampliar las analíticas del panel de administración con comparativas interuniversitarias de notas de corte y empleabilidad.

Información sobre Licencia

Información sobre Licencia

El presente documento de memoria y el código fuente del proyecto **UniHub** (Trabajo Fin de Grado desarrollado por Alejandro Ramos Rodríguez en la Universidad de Cádiz) se distribuyen bajo los términos de la licencia de código abierto **MIT License** y la licencia de documentación **Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)**.

Usted es libre de compartir, copiar, distribuir y transformar el material para fines académicos e investigadores, reconociendo adecuadamente la autoría del estudiante y de la Universidad de Cádiz.

Bibliografía

- Attard, J., Orlandi, F., Scerri, S., y Auer, S. (2015). A systematic review of open government data initiatives. *Government Information Quarterly*, 32(4):399–418.
- Batini, C., Cappiello, C., Francalanci, C., y Maurino, A. (2009). Methodologies for data quality assessment and improvement. *ACM Computing Surveys (CSUR)*, 41(3):1–52.
- Crescenzi, V., Mecca, G., y Merialdo, P. (2001). ROADRUNNER: Towards automatic data extraction from large, web sites. In *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*, pages 324–335.
- Ferrara, E., De Meo, P., Fiumara, G., y Baumgartner, R. (2014). Web data extraction, applications and techniques: A survey. *Knowledge-Based Systems*, 70:301–323.
- Glez-Peña, D., Lourenço, A., López-Fernández, H., Reboiro-Jato, M., y Fdez-Riverola, F. (2014). Web scraping technologies in an API world. *Briefings in Bioinformatics*, 15(5):788–797.
- Janssen, M., Charalabidis, Y., y Zuiderwijk, A. (2012). Benefits, adoption barriers and myths of open data and open government. *Information Systems Management*, 29(4):258–268.
- Jefatura del Estado (2023). Ley Orgánica 2/2023, de 22 de marzo, del Sistema Universitario (LOSU). Boletín Oficial del Estado, núm. 70, de 23 de marzo de 2023.
- Ministerio de Educación (2011). Real Decreto 99/2011, de 28 de enero, por el que se regulan las enseñanzas oficiales de doctorado. Boletín Oficial del Estado, núm. 35, de 10 de febrero de 2011.
- Ministerio de Universidades (2021). Real Decreto 822/2021, de 28 de septiembre, por el que se establece la organización de las enseñanzas universitarias y del procedimiento de aseguramiento de su calidad. Boletín Oficial del Estado, núm. 233, de 29 de septiembre de 2021.
- Singrodia, V., Mitra, A., y Paul, S. (2019). A review on Web Scrapping and its applications. In *2019 International Conference on Computer Communication and Informatics (ICCCI)*, pages 1–6. IEEE.
- Zhao, B. (2017). Web scraping: An efficient way of leveraging data from the web for advanced data analysis. *Research and Practice in Technology Enhanced Learning*, 12(1):1–22.

Parte IV

Anexos

A. Manual del desarrollador

A continuación se recogen las instrucciones necesarias para evolucionar el software **UniHub**. Este manual está dirigido a los desarrolladores que pretenden extender o modificar el código fuente, con el fin de incorporar nuevas funcionalidades o modificar las ya existentes.

A.1. Introducción

El proyecto **UniHub** se organiza en cuatro componentes principales situados bajo `Codigo/`: `Crawler/`, `API/`, `WWW/` y `Docker/`.

A.2. Preparación del entorno de trabajo

1. Instalar Python 3.12, Node.js 20+, PostgreSQL 15 y Docker Desktop / Engine.
2. Clonar o abrir la carpeta raíz del repositorio `d:/Proyecto`.
3. Para desarrollo en local sin Docker:
 - Entorno virtual Python del Crawler: `cd Codigo/Crawler; python -m venv .venv; ./.venv/Scripts/activate; pip install -r requirements.txt`.
 - Dependencias de la API: `pip install -r Codigo/API/requirements.txt`.
 - Dependencias Web Frontend: `cd Codigo/WWW; npm install`.

A.3. Consideraciones generales sobre el desarrollo

- **Gestión de Datos no Vacíos y Filtro Estricto:** Cualquier modificación en el crawler debe respetar el filtro estricto de titulaciones vigentes y la función `is_valid.value` de `checkpoint.py` para impedir la sobrescritura accidental de registros válidos.
- **Parser BOE:** Los cambios en `boe_pdf_parser.py` deben conservar la delimitación por titulación, la trazabilidad de la fuente y las pruebas de precisión/cobertura. Un resultado sin detalle curricular debe persistirse como incompleto, nunca completarse con valores estimados.
- **Nuevos Endpoints en la API REST:** Al incorporar nuevos endpoints en FastAPI, defina siempre los esquemas Pydantic correspondientes en `schemas/schemas.py` y añada el router a `main.py`.

- **Sistema de Diseño UCA:** Al añadir componentes visuales en React, utilice las variables CSS corporativas de la UCA (`--uca-navy`, `--uca-blue`, `--uca-cyan`, `--uca-gold`) y la clase `.badge-doctorado` definidas en `src/index.css`.
- **Resiliencia de UI:** Toda ruta pública debe estar protegida por `ErrorBoundary` y componentes pesados deben cargarse con `React.lazy` y `Suspense`. Evitar comparaciones contra datos mock en tiempo de ejecución; usar banderas de estado como `initialDataLoaded` para controlar efectos secundarios.

A.4. Instrucciones para construcción

- **Compilar Frontend (Vite Bundle):** `cd Codigo/WWW; npm run build`.
- **Construir y Levantar Todo con Docker:** `cd Codigo/Docker; docker compose up --build -d`.
- **Ejecutar Carga ETL:** `docker compose exec api python database/etl_loader.py`.

A.5. Ejecución Avanzada y Filtrado del Crawler (Fase 1)

El orquestador troncal `main_fase_1.py` dispone de una suite completa de parámetros por línea de comandos para auditorías, pruebas unitarias o ejecuciones de producción segmentadas:

- **Selección de Partes:**
 - Ejecutar solo una parte: `python main_fase_1.py --parte 4`
 - Ejecutar varias partes específicas: `python main_fase_1.py --parts 1 2`
 - Ejecutar el pipeline completo: `python main_fase_1.py --all`
- **Filtrado por Universidades (Códigos o Nombres):**
 - Por códigos RUCT: `python main_fase_1.py --parte 4 --univs 025 008`
 - Por nombres o términos clave: `python main_fase_1.py --parte 4 --univs CádizGranada"`
- **Filtrado por Titularidad Institucional:**
 - Solo públicas: `python main_fase_1.py --publicas`
 - Solo privadas: `python main_fase_1.py --privadas`
- **Filtrado Semántico por Título de Titulación:**
 - Buscar solo titulaciones de Informática: `python main_fase_1.py --grado Informática"`

- Combinar filtros: `python main_fase_1.py --parte 4 --univs 025 --grado "Medicina"`

B. Manual de usuario

Las instrucciones de uso del software se detallan a continuación. Este manual se dirige al usuario final del sistema **UniHub**.

B.1. Introducción

El Portal Web de **UniHub** (Fase 3) permite consultar el catálogo oficial de universidades de España, explorar titulaciones de Grado, Máster y Doctorado y, cuando exista una fuente curricular verificada, revisar su desglose de asignaturas, créditos ECTS y enlace al documento oficial.

B.2. Instalación

Para acceder al sistema no se requiere ninguna instalación por parte del usuario final. Únicamente se necesita un navegador web moderno (Google Chrome, Mozilla Firefox, Microsoft Edge o Safari) y conexión a la red donde esté desplegado el servicio (<http://localhost> o <http://localhost:5173>).

B.3. Uso del sistema

1. **Exploración de Universidades:** Acceda a la pestaña "Universidades" para filtrar centros públicos (listados prioritariamente en primer lugar) o privados por nombre o comunidad autónoma.
2. **Buscador de Titulaciones y BOE:** En la pestaña "Titulaciones", busque por cualquier titulación o filtre por Grado, Máster o Doctorado. Al abrir una tarjeta se muestra la estructura curricular disponible: para Grados y Másteres se desglosan las asignaturas organizadas por curso/cuatrimestre y menciones curriculares; para Doctorados (RD 99/2011) se visualiza la insignia oficial verificada, la Escuela de Doctorado tutelar, la cuadrícula de líneas de investigación científica acreditadas, actividades formativas transversales y el régimen de tutela académica anual, junto con el enlace directo a la fuente oficial. Algunos documentos oficiales sólo publican un resumen de créditos; en esos casos el portal no inventa asignaturas ausentes.
3. **Paginación Configurable:** En la parte inferior de las grillas de universidades y titulaciones, seleccione el número de elementos a mostrar por página (5, 10, 20, 50 o 100) y navegue entre páginas con los botones de control.
4. **Búsqueda por Cercanía (Geolocalización):** En la pestaña "Por Cercanía", pulse el botón "Usar Mi Ubicación Actual (GPS)" para permitir que el navegador

calcule la distancia exacta en km a cada universidad y visualícela directamente dentro de la tarjeta de cada centro.

5. **Sección "Sobre Nosotros":** Acceda a la pestaña "Sobre Nosotros" para consultar la información institucional del proyecto (Trabajo Fin de Grado de Alejandro Ramos Rodríguez en la Universidad de Cádiz).
6. **Panel del Administrador (Acceso Aislado):** Escriba la ruta manual <http://localhost/admin> en la barra de direcciones de su navegador. Introduzca la API Key definida en ADMIN_API_KEY para acceder al panel de administración. No se distribuyen credenciales por defecto.

C. Glosario de Términos y Acrónimos Técnicos

Este anexo recopila y define los principales conceptos técnicos, patrones de diseño arquitectónico y acrónimos utilizados a lo largo del desarrollo de la plataforma UniHub.

C.1. Glosario de Conceptos y Patrones de Diseño

AbortController: Interfaz estándar de la API Fetch del navegador que permite abortar y cancelar peticiones HTTP asíncronas en curso. En UniHub se utiliza para cancelar peticiones obsoletas en búsquedas con *debounce* y evitar condiciones de carrera (*race conditions*) en la actualización del estado.

Bulk Save / Bulk Insert: Estrategia de inserción masiva a nivel de ORM (SQLAlchemy) y base de datos que agrupa miles de registros en una única transacción e instrucción SQL compuesta, reduciendo el tiempo de carga ETL de 15 segundos a menos de 1,5 segundos.

Circuit Breaker (Cortocircuito): Patrón de diseño de resiliencia y estabilidad que detecta fallos consecutivos de conexión hacia servidores remotos (RUCT/BOE). Si se superan 10 errores seguidos, abre el circuito pausando el proceso durante 5 minutos para permitir la recuperación del servidor destino, abortando la universidad tras 15 minutos acumulados de caída continuada.

Code Splitting: Técnica de optimización web soportada por Vite y Webpack que divide el bundle de JavaScript en múltiples fragmentos más pequeños que se descargan bajo demanda mediante importaciones dinámicas (*React.lazy*), acelerando el tiempo de carga inicial de la página.

Core Web Vitals (CWV): Conjunto de métricas estandarizadas por Google (LCP, INP, CLS) para cuantificar la experiencia del usuario en términos de velocidad de renderizado, interactividad y estabilidad visual.

ErrorBoundary: Patrón de componente React que intercepta excepciones JavaScript no controladas en cualquier parte de su subárbol de componentes hijos, registrando el error y mostrando una interfaz gráfica de contingencia sin romper toda la aplicación.

Green IT (Tecnologías de la Información Verdes): Enfoque de ingeniería de software orientado a minimizar la huella de carbono y el consumo energético de la infraestructura computacional. En UniHub se modela a través de la telemetría de procesamiento ($\approx 0,05 \text{ gCO}_2/\text{MB}$) y el aprovechamiento de caché (99,8% de aciertos).

Healthcheck: Mecanismo de sondeo periódico configurado en Docker Compose que monitoriza el estado de operatividad de un contenedor (ej. PostgreSQL respondiendo a `pg_isready`) antes de permitir que servicios dependientes (como la API o el Crawler) comiencen a emitir peticiones.

Multi-stage build: Método de construcción de imágenes Docker que utiliza múltiples directivas `FROM` intermedias para compilar dependencias pesadas y copiar únicamente los binarios estáticos finales a una imagen mínima en producción (`alpine`), reduciendo el peso de cientos de megabytes a menos de 25 MB.

Patrón Productor-Consumidor: Arquitectura de concurrencia desacoplada donde un proceso productor (descarga de red multihilo I/O) introduce tareas en una cola thread-safe (`multiprocessing.Queue`) y un conjunto de procesos consumidores (trabajadores CPU) procesan el parseo sintáctico de los PDFs en paralelo.

React.lazy y Suspense: APIs nativas de React para carga perezosa de componentes que permiten diferir la importación de vistas pesadas (como el panel de administración o el simulador financiero) hasta el momento exacto en que el usuario navega a dichas secciones.

Robots Exclusion Protocol (RFC 9309): Estándar web que regula las directivas de acceso de los rastreadores automatizados mediante el archivo `robots.txt` y el parámetro de cortesía `Crawl-delay`.

SQLite WAL (Write-Ahead Logging): Modo de diario transaccional de SQLite implementado en `unihub_cache.sqlite3` que permite lecturas concurrentes instantáneas ($< 0,1$ ms) sin ser bloqueadas por operaciones de escritura simultáneas.

UseMemo y useCallback: Ganchos (*hooks*) de optimización en React que memorizan cálculos computacionalmente pesados y referencias de funciones para prevenir re-renderizados innecesarios del árbol del DOM.

D. Esquema Físico DDL de Base de Datos

En este apéndice se proporciona la definición formal completa del esquema relacional físico de la base de datos PostgreSQL de **UniHub** (`schema.sql`). Incluye la creación de tablas principales, claves foráneas con restricciones de integridad referencial y borrado en cascada, tipos semiestructurados (**JSONB**), índices de búsqueda acelerada mediante trigramas (`pg_trgm`) y la parametrización del rol de solo lectura de la API REST (`unihub_api_user`).

Extracto de código D.1: Esquema DDL completo de PostgreSQL (`schema.sql`)

```
--
=====

-- ESQUEMA BASE DE DATOS POSTGRESQL - UNIHUB
--
=====

-- 1. Tabla de Universidades (Publicas y Privadas)
CREATE TABLE IF NOT EXISTS universidades (
    id SERIAL PRIMARY KEY,
    codigo VARCHAR(10) UNIQUE NOT NULL,
    nombre VARCHAR(500) NOT NULL,
    tipo VARCHAR(100),
    comunidad_autonoma VARCHAR(200),
    municipio VARCHAR(200),
    provincia VARCHAR(200),
    web VARCHAR(500),
    email VARCHAR(500),
    telefono VARCHAR(200),
    gestionado_por_admin BOOLEAN DEFAULT FALSE,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Tabla de Titulaciones (Grados, Masteres, Doctorados
    Vigentes)
CREATE TABLE IF NOT EXISTS titulaciones (
    id SERIAL PRIMARY KEY,
    codigo_estudio VARCHAR(20) UNIQUE NOT NULL,
    titulo TEXT NOT NULL,
    nivel_academico TEXT,
    estado VARCHAR(200),
```

```

universidad_codigo VARCHAR(10) NOT NULL REFERENCES
    universidades(codigo) ON DELETE CASCADE,
precio_credito_ects NUMERIC(6, 2),
precio_credito_2 NUMERIC(6, 2),
precio_credito_3 NUMERIC(6, 2),
precio_credito_4 NUMERIC(6, 2),
precio_estimado_anual NUMERIC(8, 2),
fuente_precio VARCHAR(255),
centro_adscrito TEXT,
es_alianza_europea BOOLEAN NOT NULL DEFAULT FALSE,
web_fuente_directa_url TEXT,
gestionado_por_admin BOOLEAN DEFAULT FALSE,
creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Tabla de Planes de Estudio (Metadatos BOE por
    Titulacion)
CREATE TABLE IF NOT EXISTS planes_estudio (
    id SERIAL PRIMARY KEY,
    codigo_estudio VARCHAR(20) UNIQUE NOT NULL REFERENCES
        titulaciones(codigo_estudio) ON DELETE CASCADE,
    boe_url TEXT,
    boe_fecha DATE,
    origen_fuente VARCHAR(100),
    pdf_sha256 VARCHAR(64),
    estado_calidad VARCHAR(64) NOT NULL DEFAULT '
        pendiente_revision',
    motivos_calidad JSONB,
    fuente_verificada_url TEXT,
    verificado_en TIMESTAMP,
    fecha_procesado TIMESTAMP,
    tipo_estructura VARCHAR(100),
    ects_exigidos TEXT,
    programa_doctoral JSONB,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Tabla de Resumen de Creditos ECTS
CREATE TABLE IF NOT EXISTS resumen_creditos (
    id SERIAL PRIMARY KEY,
    plan_estudio_id INT NOT NULL REFERENCES planes_estudio(id
        ) ON DELETE CASCADE,
    tipo_credito VARCHAR(200) NOT NULL,
    cantidad_creditos VARCHAR(50) NOT NULL
);

```

```

-- 5. Tabla de Elementos Curriculares (Asignaturas, Modulos,
Materias)
CREATE TABLE IF NOT EXISTS elementos_curriculares (
    id SERIAL PRIMARY KEY,
    plan_estudio_id INT NOT NULL REFERENCES planes_estudio(id
    ) ON DELETE CASCADE,
    codigo VARCHAR(50),
    nombre_elemento TEXT NOT NULL,
    modulo TEXT,
    materia TEXT,
    creditos_ects NUMERIC(5, 2),
    caracter VARCHAR(100),
    curso INT,
    cuatrimestre VARCHAR(50),
    idioma VARCHAR(100),
    departamento TEXT,
    creditos_teoria NUMERIC(5, 2),
    creditos_practica NUMERIC(5, 2),
    url_guia_docente TEXT,
    guia_docente JSONB,
    fuente_elemento VARCHAR(50) DEFAULT 'BOE',
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 6. Tabla de Registro de Errores e Incidencias del Crawler
CREATE TABLE IF NOT EXISTS errores_crawler (
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    universidad_codigo VARCHAR(10),
    url_afectada TEXT,
    codigo_http INT,
    mensaje_error TEXT,
    contexto JSONB
);

-- 7. Tabla de Metricas de Rendimiento de Ingesta
CREATE TABLE IF NOT EXISTS estadisticas_rendimiento (
    id SERIAL PRIMARY KEY,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fase_crawler VARCHAR(50),
    total_titulaciones_procesadas INT,
    total_planes_completos INT,
    total_planes_parciales INT,
    duracion_segundos NUMERIC(8, 2),
    memoria_rss_mb NUMERIC(8, 2),
    tasa_acierto_porcentaje NUMERIC(5, 2)
);

```



```

--
=====

-- INDICES OPTIMIZADOS PARA BUSQUEDA Y RENDIMIENTO
--
=====

CREATE INDEX IF NOT EXISTS idx_titulaciones_univ_codigo ON
    titulaciones(universidad_codigo);
CREATE INDEX IF NOT EXISTS idx_titulaciones_nivel ON
    titulaciones(nivel_academico);
CREATE INDEX IF NOT EXISTS idx_elementos_plan_id ON
    elementos_curriculares(plan_estudio_id);
CREATE INDEX IF NOT EXISTS idx_elementos_curso ON
    elementos_curriculares(curso);

-- Extension pg_trgm para búsquedas textuales difusas en
    español
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX IF NOT EXISTS idx_univ_nombre_trgm ON
    universidades USING gin (nombre gin_trgm_ops);
CREATE INDEX IF NOT EXISTS idx_tit_titulo_trgm ON
    titulaciones USING gin (titulo gin_trgm_ops);

--
=====

-- CONFIGURACION DE SEGURIDAD RBAC: USUARIO DE SOLO LECTURA
    API
--
=====

DO
$do$
BEGIN
    IF NOT EXISTS (
        SELECT FROM pg_catalog.pg_roles
        WHERE rolname = 'unihub_api_user') THEN
        CREATE ROLE unihub_api_user WITH LOGIN PASSWORD '${
            POSTGRES_API_PASSWORD}';
    END IF;
END
$do$;

GRANT CONNECT ON DATABASE unihub_db TO unihub_api_user;
GRANT USAGE ON SCHEMA public TO unihub_api_user;

```

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO  
unihub_api_user;  
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON  
TABLES TO unihub_api_user;
```